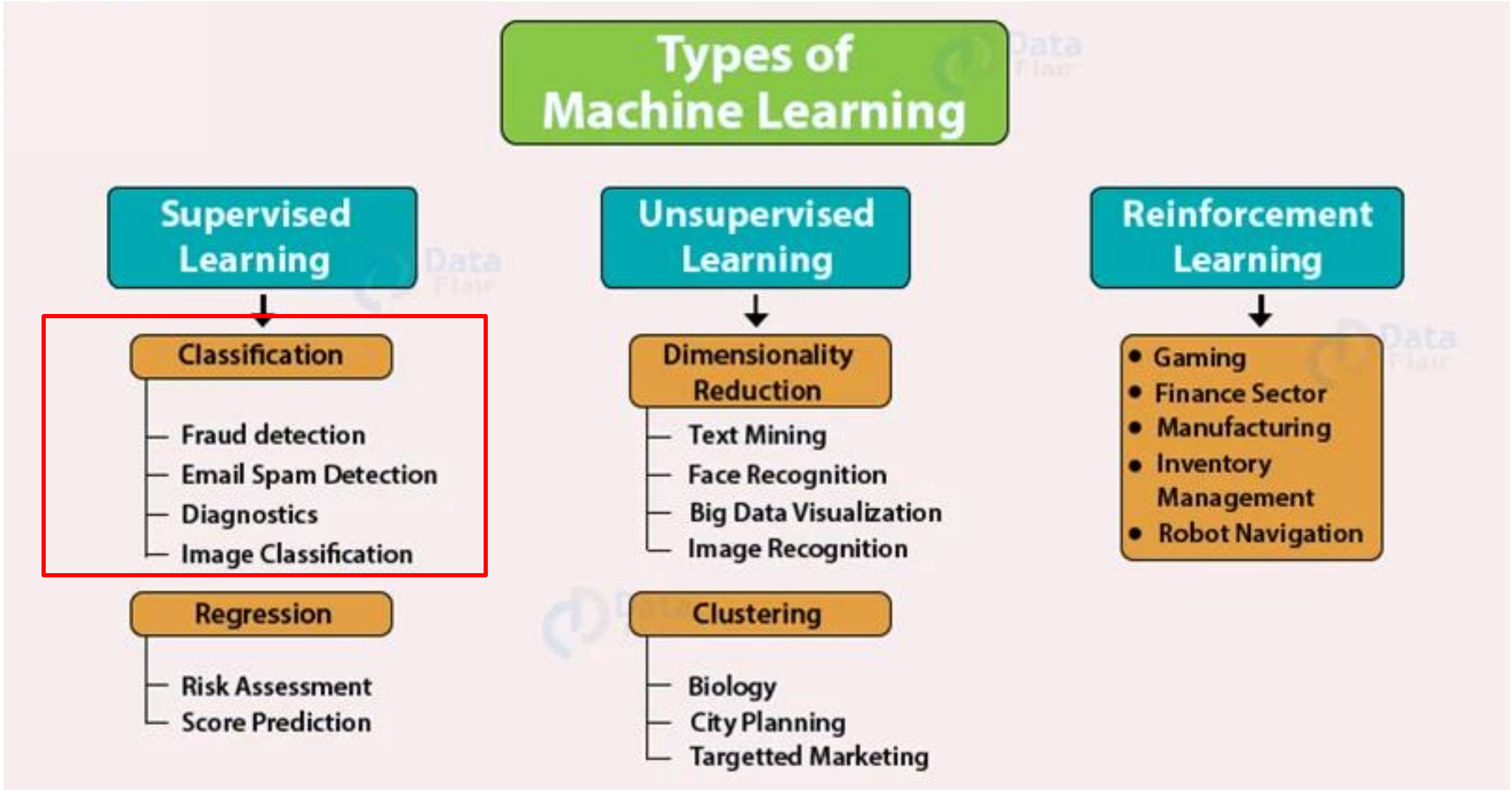
Classification

Our DS Process

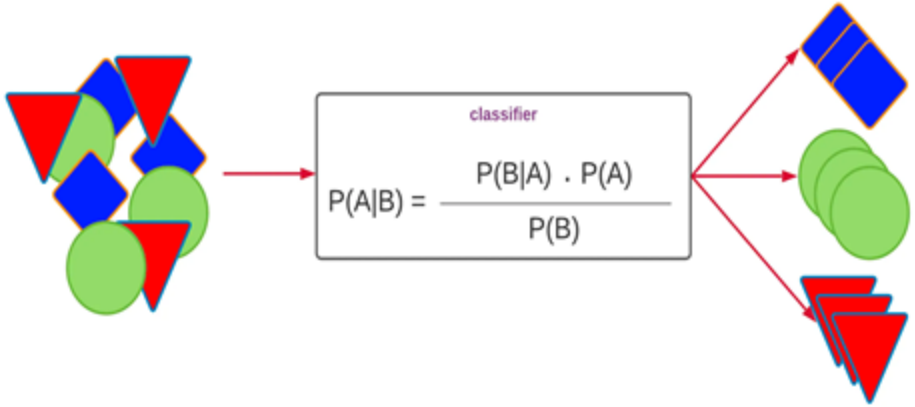
1. **Acquire data**
2. **Explore data**
3. **Clean data**
4. **Feature engineer**
5. **Train**
6. **Test**
7. **Iterate**
8. **Present**


Types of Machine Learning
Supervised Learning
Classification
Fraud detection
Email Spam Detection
Diagnostics
Image Classification
Regression
Risk Assessment
Score Prediction
Unsupervised Learning
Dimensionality Reduction
Text Mining
Face Recognition
Big Data Visualization
Image Recognition
Clustering
Biology
City Planning
Targetted Marketing
Reinforcement Learning
Gaming
Finance Sector
Manufacturing
Inventory Management
Robot Navigation

Different Classification Methods:

- Naive Bayes
- Support Vector Machines
- Random Forest

(1)Naive Bayes Classifiers



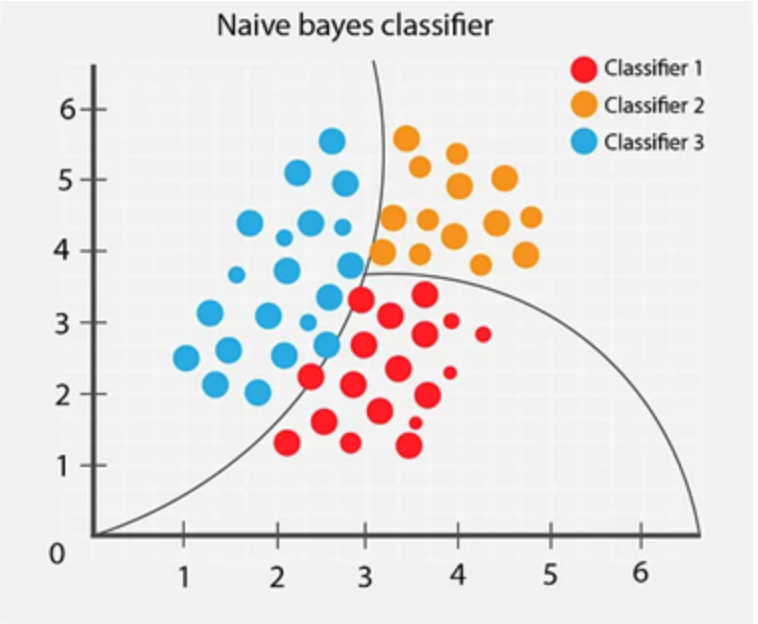
Naive Bayes Algorithm

A probabilistic classification algorithm based on Bayes' Theorem. It assumes that the features are independent given the class label. This assumption of independence is what makes it "naive."

$$P(A|B) = \frac{P(B|A)\ P(A)}{P(B)}$$

using Bayesian probability terminology, the above equation can be written as

$$\text{Posterior} = \frac{\text{prior x likelihood}}{\text{evidence}}$$



Naive Bayes Algorithm

$$P(y|X) = \frac{P(X|y)P(y)}{P(X)}$$

Where: Variable y represents the class labels.
Variable X represent the features.

$$X = (x_1, x_2, x_3, ..., x_n)$$

By substituting for X and expanding using the chain rule we get,

$$P(y|x_1, ..., x_n) = \frac{P(x_1|y)P(x_2|y)...P(x_n|y)P(y)}{P(x_1)P(x_2)...P(x_n)}$$

Since the denominator is constant across all data points, we can ignore it, simplifying the calculation to proportionality.

$$P(y|x_1, ..., x_n) \propto P(y) \prod_{i=1}^{n} P(x_i|y)$$

we need to **find the class y with maximum probability**.

Final Decision Rule:

$$y = argmax_y P(y) \prod_{i=1}^{n} P(x_i|y)$$

Assumptions of Naive Bayes Algorithm

- **Independence Assumption:** All features X1,X2,…., Xn are independent given the class y.

$$P(X|y) = P(X_1|y) \cdot P(X_2|y) \cdots P(X_n|y)$$

- **Features are equally important:** Treat all features X1,X2, … Xn as they contribute equally to the prediction of the class label.

- **Continuous features are assumed to be normally distributed:** within each class (Gaussian Naive Bayes).
- **No missing data**

Example: Assumptions of Naive Bayes Algorithm

- **Task:** Classify if a day is suitable for playing golf based on features.
- **Features:** Columns represent features; rows represent individual entries.

Assumptions:

- **Independence:** Hot temperature does not imply high humidity.
- **Equal Impact:** Windy conditions do not outweigh other factors in the decision to play golf.

	OUTLOOK	TEMPERATURE	HUMIDITY	WINDY	PLAY GOLF
0	Rainy	Hot	High	False	No
1	Rainy	Hot	High	True	No
2	Overcast	Hot	High	False	Yes
3	Sunny	Mild	High	False	Yes
4	Sunny	Cool	Normal	False	Yes
5	Sunny	Cool	Normal	True	No
6	Overcast	Cool	Normal	True	Yes
7	Rainy	Mild	High	False	No
8	Rainy	Cool	Normal	False	Yes
9	Sunny	Mild	Normal	False	Yes
10	Rainy	Mild	Normal	True	Yes
11	Overcast	Mild	High	True	Yes
12	Overcast	Hot	Normal	False	Yes
13	Sunny	Mild	High	True	No

Consider the problem of playing golf.

Uses of Naive Bayes Algorithm

Classification problems; commonly used in **text classification** within Natural Language Processing.

- Why? Text data has high dimensionality, with each word represent one feature (e.g., spam filtering, sentiment analysis).

Process:

- **Compute Probabilities:** Calculate probabilities for each label/tag based on features.
- **Select Best Label:** Choose the label with the highest calculated probability.
- **Apply Bayes' Theorem:** Utilize Bayes' Theorem, incorporating prior knowledge of the features.

Example: Naive Bayes

TASK: Predict the label ("Sports" or "Non-Sports") for the new text data "A very close game". Is it categorized as "Sports" or "Non-Sports"?

- **P (Sports | A very close game)**
- **P (Not sports | A very close game)**

Steps: Calculate Prior Probabilities:

P(Sports) = 3/5 ; P(Not Sports) = 2/5

Assume you have the following documents (text) and their associated tags.

- Each word (token) in the document is treated as a feature (Feature Representation →LATER TOPIC).

Text	Tag
"A great game"	Sports
"The election was over"	Not sports
"Very clean match"	Sports
"A clean but forgettable game"	Sports
"It was a close election"	Not sports

Training Data

Example: Naive Bayes

P (Sports | A very close game)
P (Not sports | A very close game)

Steps Likelihood: We want to count how many times **the sentence "A very close game" appears** in the Sports or Non-Sports tag

We compare:

Text	Tag
"A great game"	Sports
"The election was over"	Not sports
"Very clean match"	Sports
"A clean but forgettable game"	Sports
"It was a close election"	Not sports

Training Data

$$P(\text{Sports}|\text{A very close game}) = \frac{P(\text{A very close game}|\text{Sports}) \cdot P(\text{Sports})}{P(\text{A very close game})}$$

and

$$P(\text{Not Sports}|\text{A very close game}) = \frac{P(\text{A very close game}|\text{Not Sports}) \cdot P(\text{Not Sports})}{P(\text{A very close game})}$$

Example: Naive Bayes

P (Sports | A very close game)
P (Not sports | A very close game)

Steps Likelihood: We want to count how many times **the sentence "A very close game" appears** in the Sports or Non-Sports tag

Text	Tag
"A great game"	Sports
"The election was over"	Not sports
"Very clean match"	Sports
"A clean but forgettable game"	Sports
"It was a close election"	Not sports

Training Data

Decision Criterion: To classify, we check which is greater:

- If $P(\text{A very close game}|\text{Sports}) \cdot P(\text{Sports}) > P(\text{A very close game}|\text{Not Sports}) \cdot P(\text{Not Sports})$, then classify as Sports.
- Otherwise, classify as Not Sports.

Did you see any issue?

Being Naive: Here comes the Naive Part!

Naive part: Naive Bayes assume that every word (features) in a sentence is conditionally independent of the other ones given the class. This means that we're no longer looking at entire sentences, but rather at individual words.

We write this as:

$$P(a\,very\,close\,game) = P(a) \times P(very) \times P(close) \times P(game)$$

Each word is treated as a separate feature that contributes to the overall classification decision.

Back to Example: Naive Bayes

P (Sports | A very close game)
P (Not sports | A very close game)

Steps Likelihood: We want to count how many times **the sentence "A very close game" appears** in the Sports or Non-Sports tag

Calculate Probabilities of each individual word :

Text	Tag
"A great game"	Sports
"The election was over"	Not sports
"Very clean match"	Sports
"A clean but forgettable game"	Sports
"It was a close election"	Not sports

Training Data

$$P(\text{A very close game}|\text{Sports}) = P(A|\text{Sports}) \cdot P(\text{very}|\text{Sports}) \cdot P(\text{close}|\text{Sports}) \cdot P(\text{game}|\text{Sports})$$

Example: $$P(\text{game}|\text{Sports}) = \frac{\text{Count of "game" in Sports}}{\text{Total words in Sports}} = 2/11$$

Did you see any issue?

Back to Example: Naive Bayes

P (Sports | A very close game)
P (Not sports | A very close game)

Steps Likelihood: We want to count how many times **the sentence "A very close game" appears** in the Sports or Non-Sports tag

Calculate Probabilities of each individual word :

Text	Tag
"A great game"	Sports
"The election was over"	Not sports
"Very clean match"	Sports
"A clean but forgettable game"	Sports
"It was a close election"	Not sports

Training Data

$$P(\text{A very close game}|\text{Sports}) = P(A|\text{Sports}) \cdot P(\text{very}|\text{Sports}) \cdot 0 \cdot P(\text{game}|\text{Sports})$$

Solution: Laplace smoothing: we add 1 to every count so it's never zero.

Laplace / Additive smoothing

A technique in Naive Bayes that **addresses zero probabilities when a word is absent** from training data.

- Adds a small constant alpha (usually 1) to the numerator and the number of unique words to the denominator, ensuring that every word has a non-zero probability.

Text	Tag
"A great game"	Sports
"The election was over"	Not sports
"Very clean match"	Sports
"A clean but forgettable game"	Sports
"It was a close election"	Not sports

Training Data

Formula: For a word w in class C:

$$P(w|C) = \frac{\text{Count}(w, C) + 1}{\text{Total words in } C + V}$$

Where:

- $\text{Count}(w, C)$ = Number of times word w appears in class C.
- V = Total number of unique words (vocabulary size).
- Adding 1 to the count prevents any probability from being zero.

['a', 'great', 'very', 'over', 'it', 'but', 'game', 'election', 'clean', 'close', 'the', 'was', 'forgettable', 'match'].

V= number of possible words = 14

Full Solution using Laplace smoothing

words	P(word/sports)	P(word/not sports)
a	(2 + 1) / (11 + 14) = 0.12	(1 + 1) / (9 + 14) = 0.0869
very	(1 + 1) / (11 + 14) = 0.08	(0 + 1) / (9 + 14) = 0.0437
close	(0 + 1) / (11 + 14) = 0.04	(1 + 1) / (9 + 14) = 0.0869
game	(2 + 1) / (11 + 14) = 0.12	(0 + 1) / (9 + 14) = 0.0437

Now we just multiply all the probabilities, and see who is bigger:

P(sports | a very close game) = P(a|sports) * P(very|sports) * P(close|sports) * P(game|sports) * P(sports) = 0.12 * 0.08 * 0.04 * 0.12 * (3/5) = 0.0000276

P(not sports | a very close game) = P(a|not sports) * P(very|not sports) * P(close|not sports) * P(game|sports) * P(sports) = 0.0869 * 0.0437 * 0.0869 * 0.0437 * (2/5) = 0.00000572

From this clearly P(sports | a very close game)> P(not sports | a very close game)

Excellent! Our classifier gives "A very close game" the **Sports** tag.

Text	Tag
"A great game"	Sports
"The election was over"	Not sports
"Very clean match"	Sports
"A clean but forgettable game"	Sports
"It was a close election"	Not sports

Training Data

Python Implementation for Naive Bayes using Scikit Learn

```
from sklearn.naive_bayes import CategoricalNB

# Initialize the Categorical Naive Bayes Classifier with Laplace smoothing
cn_model = CategoricalNB(alpha=1.0)

# Fit the model, predict, and evaluate as shown earlier
```

```
# load the iris dataset
from sklearn.datasets import load_iris
iris = load_iris()

# store the feature matrix (X) and response vector (y)
X = iris.data
y = iris.target

# splitting X and y into training and testing sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4,
random_state=1)

# training the model on training set
from sklearn.naive_bayes import GaussianNB
gnb = GaussianNB()
gnb.fit(X_train, y_train)

# making predictions on the testing set
y_pred = gnb.predict(X_test)

# comparing actual response values (y_test) with predicted response values (y_pred)
from sklearn import metrics
print("Gaussian Naive Bayes model accuracy(in %):", metrics.accuracy_score(y_test,
y_pred)*100)
```

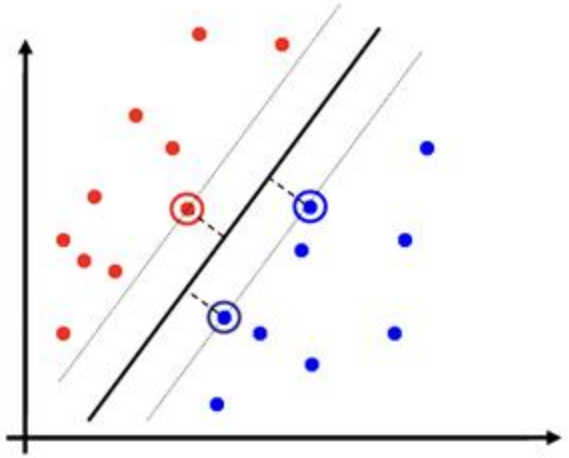
(2) Support Vector Machine (SVMs)

Support Vector Machines

A supervised ML algorithm used for both regression and classification tasks. Aim to find the best possible decision boundary (identify a hyperplane) that separates data points into different classes.

- SVMs (Vapnik, 1990's) choose the linear separator with the **largest margin**

Robust to outliers!



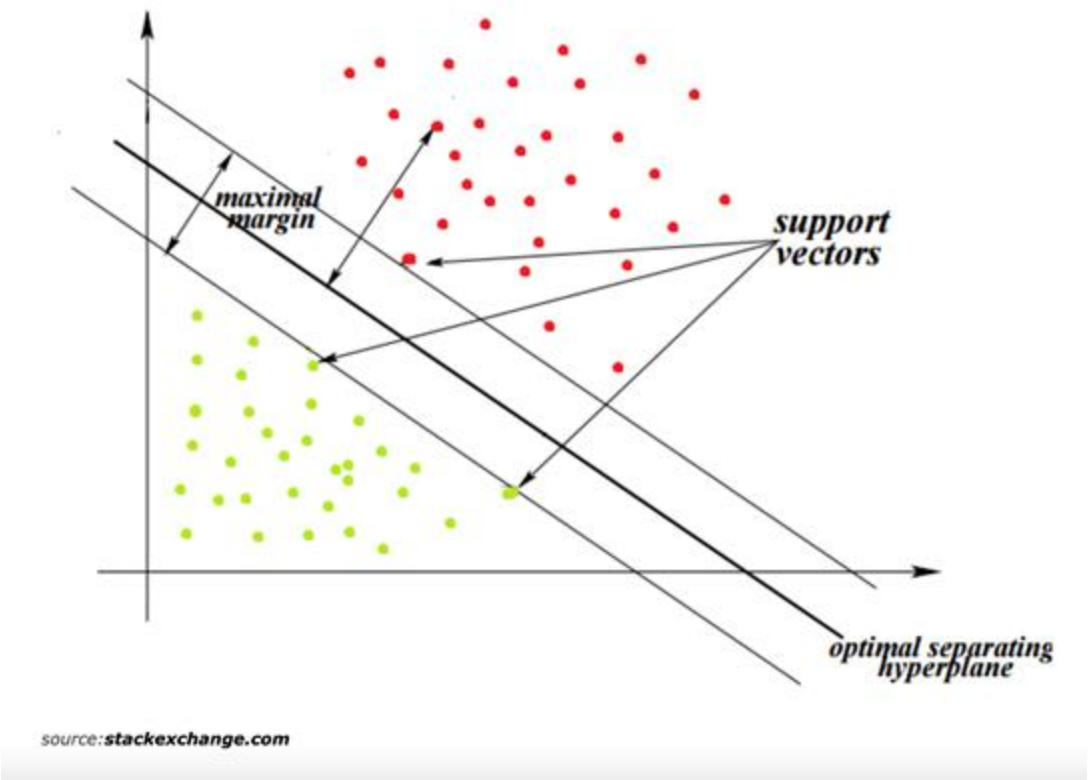


V. Vapnik

- Good according to intuition, theory, practice

Support Vector Machines

Objective: find a optimal hyperplane in an N-dimensional space(N — the number of features) that separates different classes in the feature space.

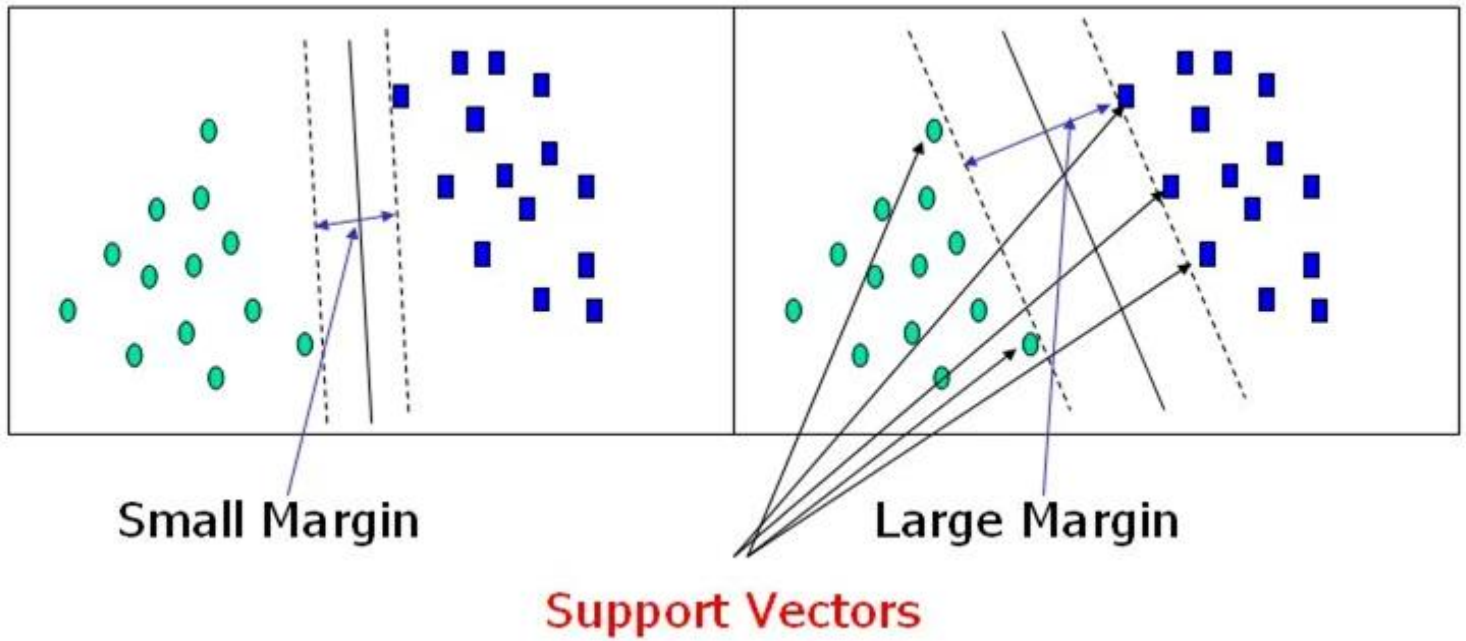


Divides data based on the maximum margin hyperplane.

What is Support Vectors?: Data points closest to the hyperplane that influence its position and orientation. These points are critical for the model's decision boundary.

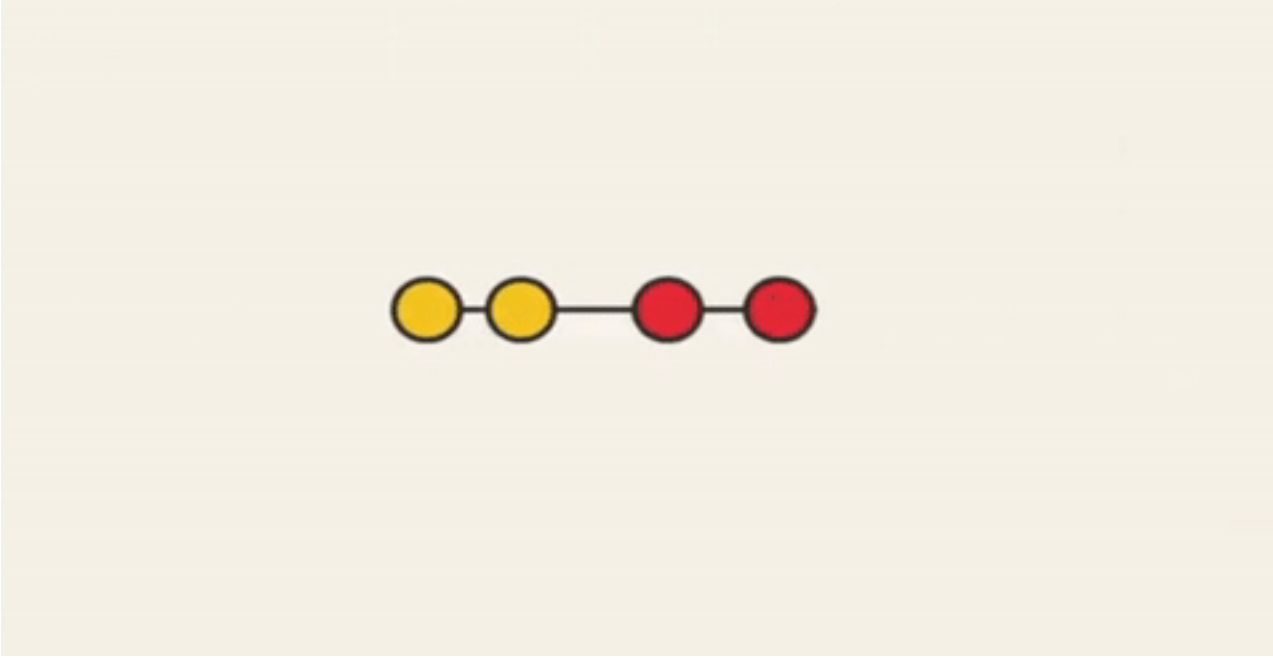
What is Support Vector

The **closest data points to the hyperplane**, which makes a critical role in deciding the hyperplane and margin (distance between the support vector and hyperplane.). They are the only instances that determine, or support, the hyperplane. It influence the position and orientation of the hyperplane.



What is Hyperplane?

Hyperplanes are decision boundaries that help classify the data points. It is is a flat surface that is one dimension lower than the input feature space.



In a two-dimensional feature space, a hyperplane is a line like y=mx+b.

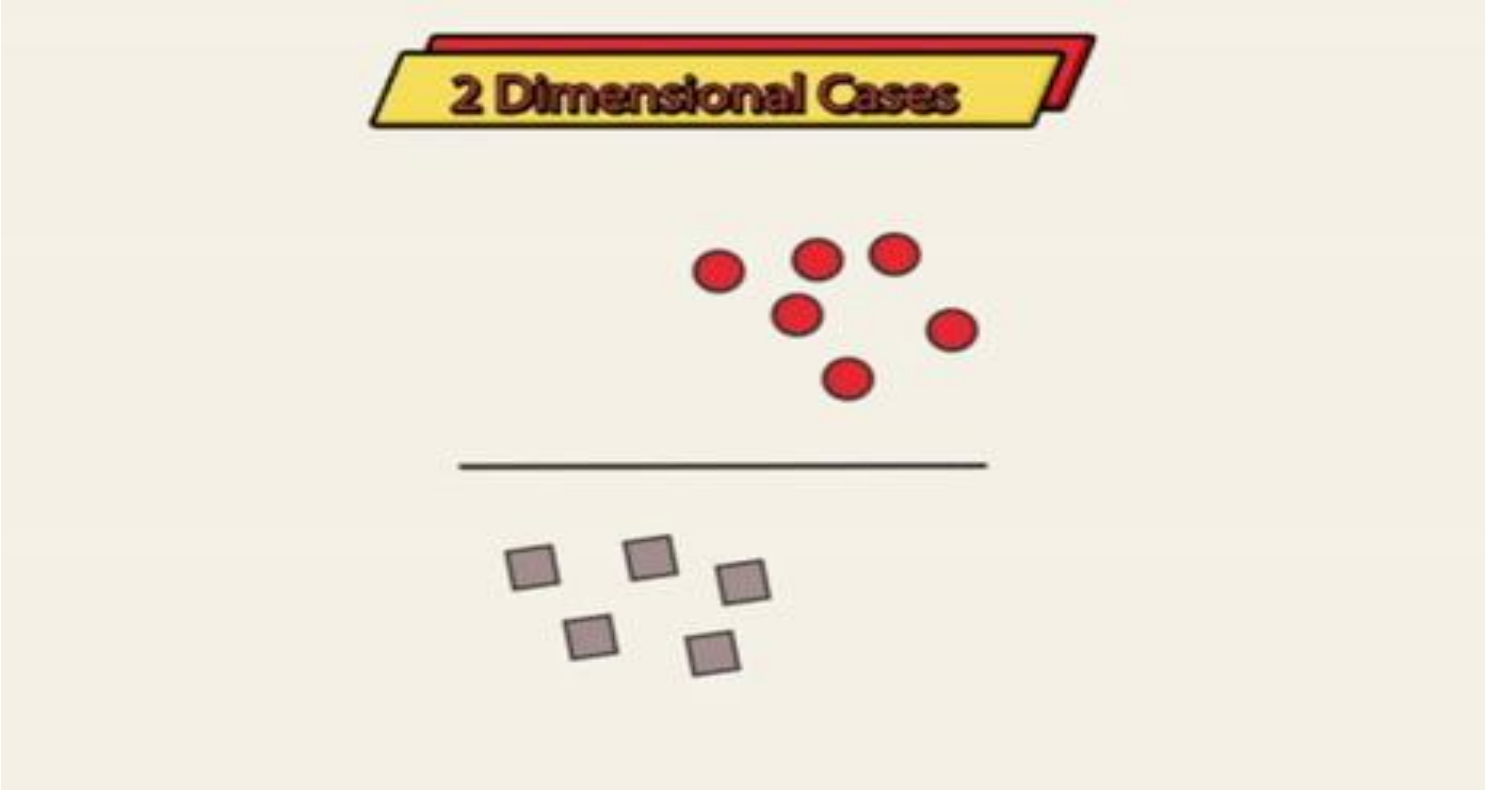
In a three-dimensional feature space, a hyperplane is a plane like x+3y-2z=5.

Intuitively, an SVM hyperplane separates the feature space into two regions. (Ref: zybook)

dimension of the hyperplane depends upon the number of features.

Back to: Support Vector Machines

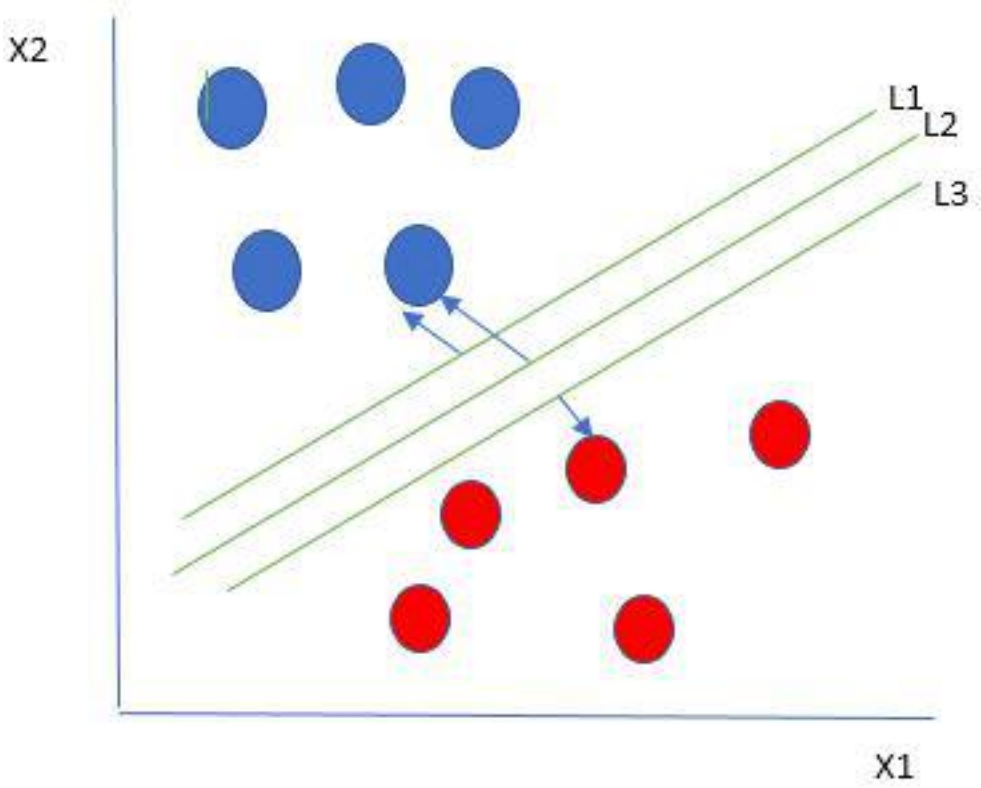
Consider 2-dimensional class: (many possible hyperplanes could be chosen)



Objective: find a optimal hyperplane that has **the maximum margin,** i.e the maximum distance between data points of both classes.

What is Margin in Support Vector Machines

Margin: The distance between the hyperplane and the nearest support vectors.
SVM aims to maximize this margin for better generalization.



Multiple hyperplanes separate the data from two classes.

Q: Which one to choose?

One reasonable choice as the best hyperplane is the one that represents the **largest separation or margin** between the two classes.

Why maximizing the margin?

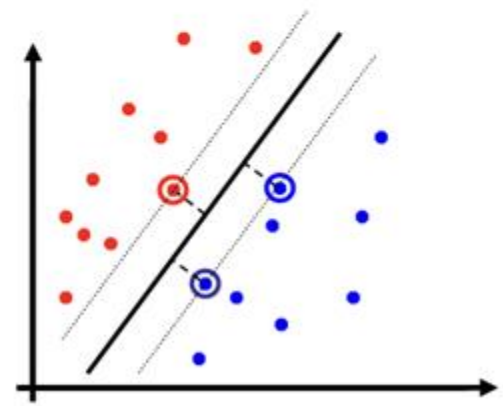
helps create a wider gap between data points of different classes, making the model better at classifying new data by allowing a larger buffer zone between categories.

Support vector machines: 3 key ideas

1. Use **optimization** to find solution (i.e. a hyperplane) with few errors

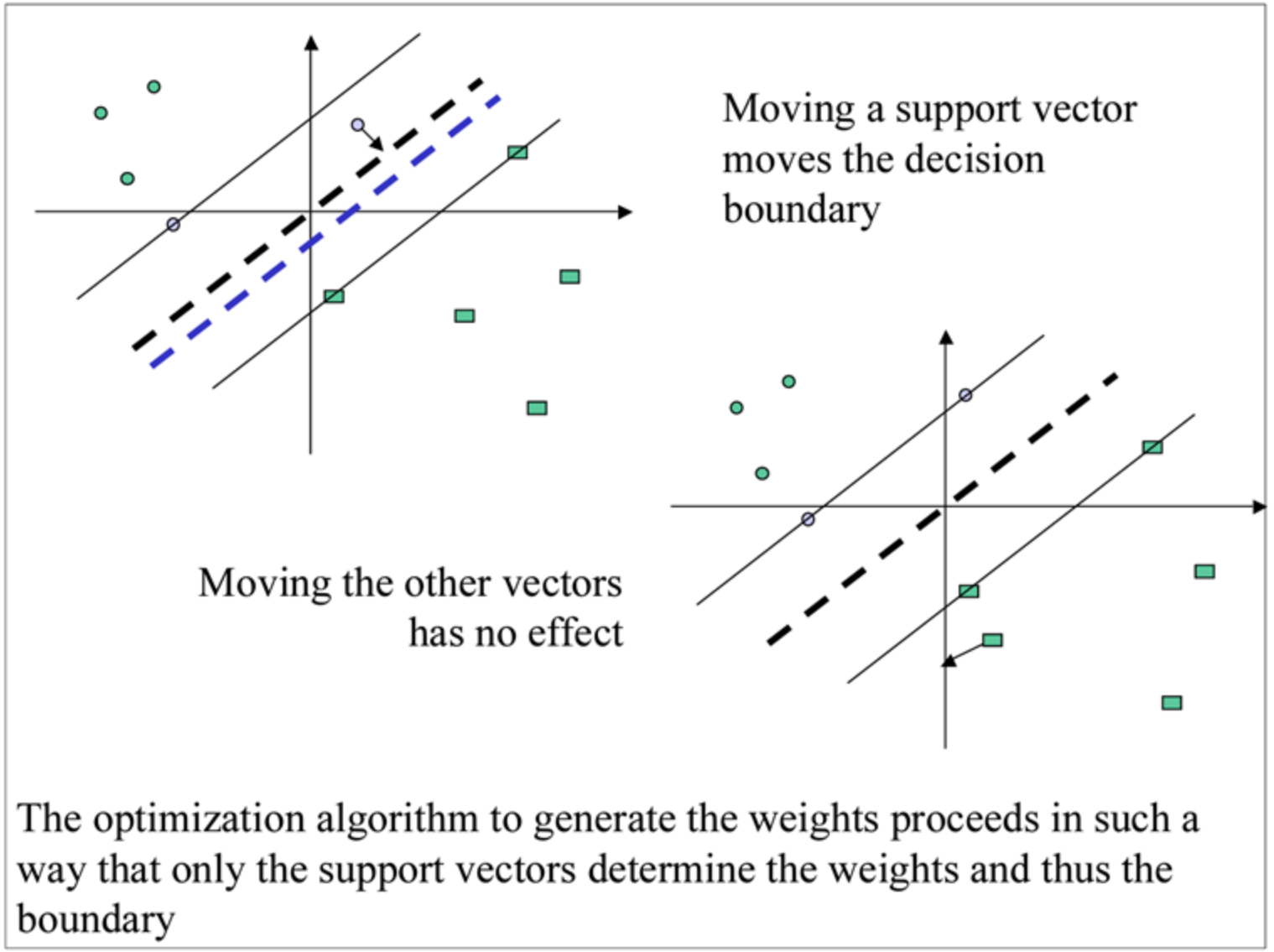
2. Seek **large margin** separator to improve generalization

3. Use **kernel trick** to make large feature spaces computationally efficient



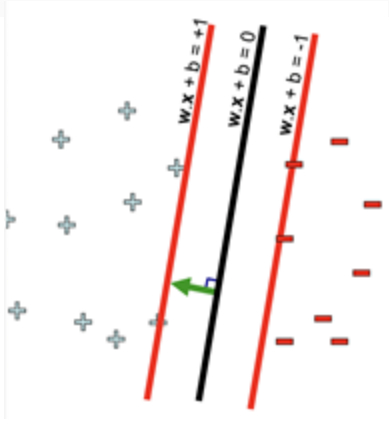
Remember

Support vectors are the critical elements of the training set that **can (or may)** change the position of the dividing hyperplane if removed.



SVM: Loss function

In the SVM algorithm, we are looking to maximize the margin between the data points and the hyperplane. The loss function that helps maximize the margin is hinge loss.



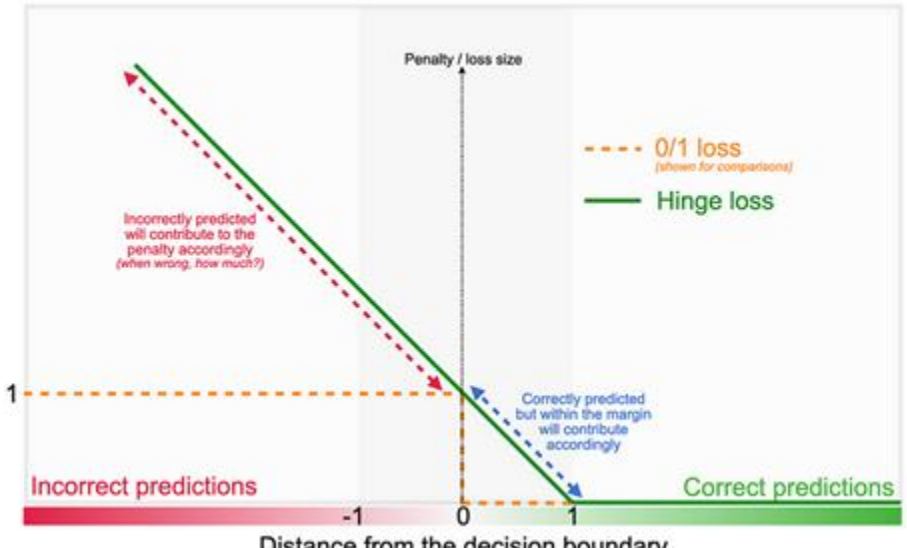
In SVMs, the decision function $f(x)$ is typically represented as:

$$f(x) = w \cdot x + b$$

The hinge loss function can be expressed as:

$$L(w, b) = \sum_{i=1}^{N} \max(0, 1 - y_i \cdot f(x_i))$$

Where
y = True class label (lets say, +1 or -1
f(x) = predicted output label for the given input X



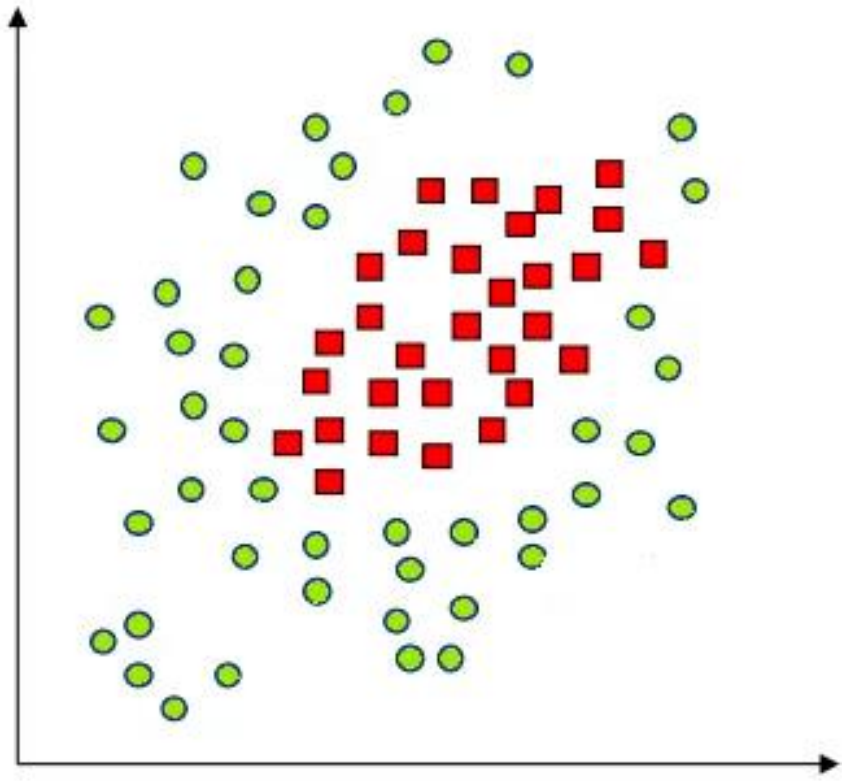
Where

$$L(y_i, f(x_i)) = \begin{cases} 0 & \text{if } y_i \cdot f(x_i) \geq 1 \\ 1 - y_i \cdot f(x_i) & \text{if } y_i \cdot f(x_i) < 1 \end{cases}$$

how far a particular data point is from being correctly classified

However

Not all data are linearly separable (meaning there is no single hyperplane that can perfectly separate the



When data isn't separable by a straight line, SVMs use a clever math trick → **“Kernel Trick”**

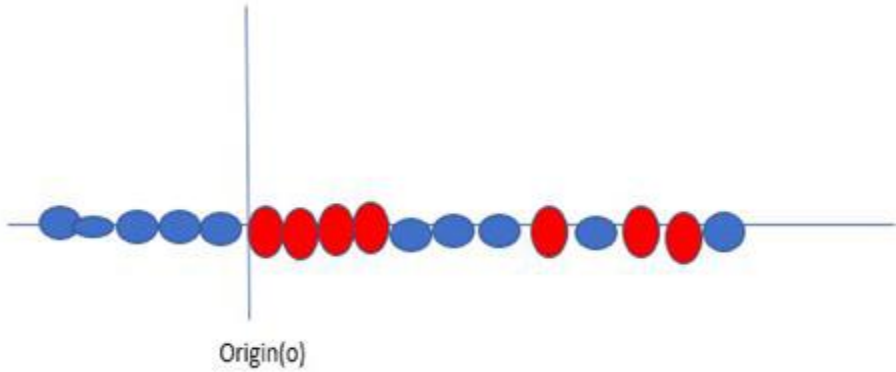
Kernel Trick:

Mathematical method used to transform the original data into a higher-dimensional space where the data becomes linearly separable.

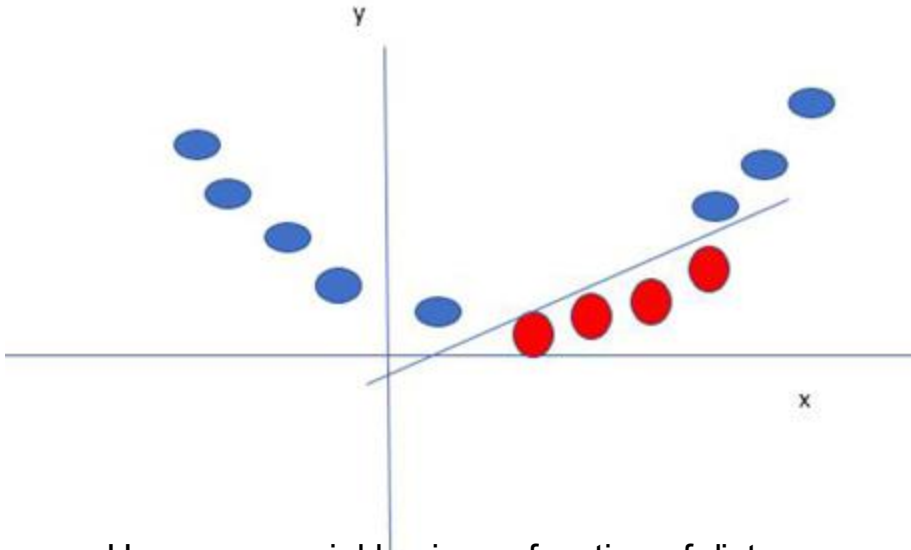
- This mapping uses a kernel function, which computes the inner products between data points in the higher-dimensional space directly, instead of explicitly calculating their coordinates.
- Popular kernel functions include:

- **Linear Kernel:** $K(x_i, x_j) = x_i \cdot x_j$
- **Polynomial Kernel:** $K(x_i, x_j) = (x_i \cdot x_j + c)^d$
- **Radial Basis Function (RBF) Kernel:** $K(x_i, x_j) = e^{-\gamma \|x_i - x_j\|^2}$
- **Sigmoid Kernel:** $K(x_i, x_j) = \tanh(\alpha x_i \cdot x_j + c)$

https://medium.com/@zxr.nju/what-is-the-kernel-trick-why-is-it-important-98a98db0961d



Mapping 1D data to 2D to become able to separate the two classes using Kernel



Here, new variable yi as a function of distance from origin o

Python Implementation for SVM using Scikit Learn

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# Load the Iris dataset
data = load_iris()
X = data.data  # Features
y = data.target  # Target labels

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_sta

# Initialize the SVM Classifier
svm_model = SVC(kernel='linear', random_state=42)  # You can change the kernel as

# Fit the model to the training data
svm_model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = svm_model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

# Print results
print(f"Accuracy of SVM: {accuracy:.2f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Next: Random Forest Classifier

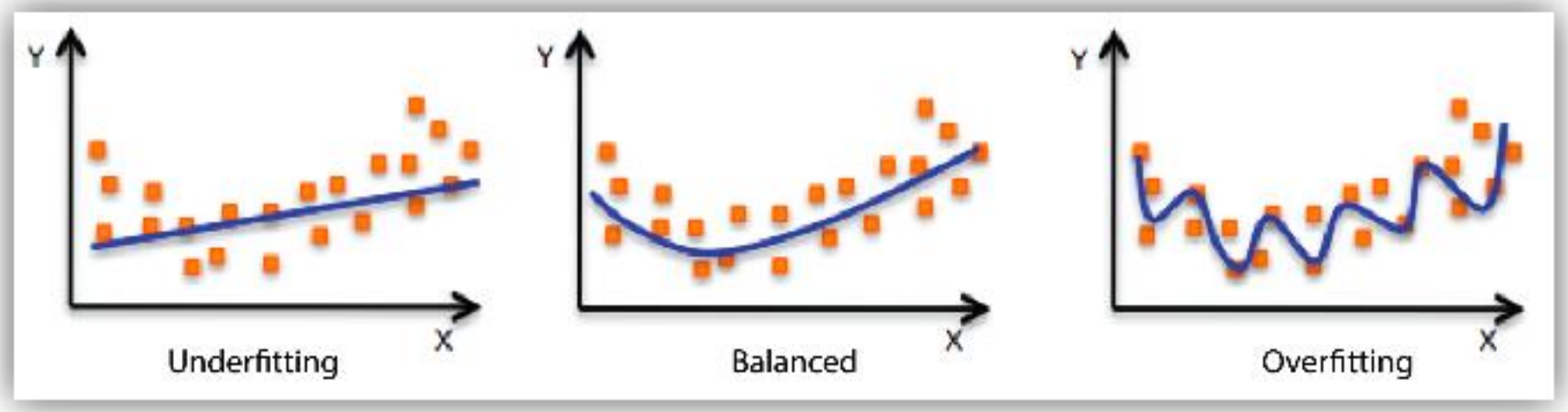
But before that, let’s talk about “Overfitting” and “Underfitting” a bit.

Recap: We split dataset into training and testing before before training and evaluating a machine learning model. But why do we test?

We don't want to use a model that doesn't work! There are many reasons a model might break, but the main ones are:

- **Overfitting**
- **Underfitting**

Overfitting and **Underfitting** are two crucial concepts in machine learning and are the prevalent ***causes for the poor performance*** of a machine learning model.



Combatting Training Failures

Overfitting

Do we think decision trees are more prone to over or under-fitting?

What about KNN where K=1? Or K=N?

Overfitting - in KNN

In KNN k=1 (or small value of k): Prone to overfitting.

k=1: Overfitting (too specific): With a small k, the model becomes highly sensitive to noise and outliers in the training data.

- It **reacts too strongly to individual points**, including noise.
- It **memorizes training data points**, capturing noise instead of learning general patterns.
- This results in **poor performance on new, unseen data** as it fails to filter out irrelevant information.

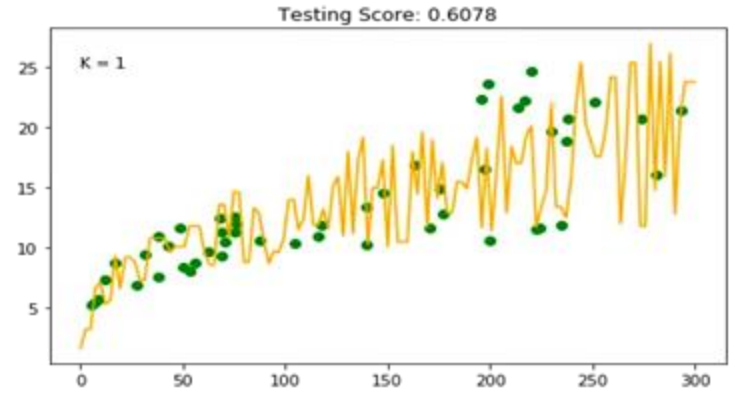
But in KNN

K=N: Underfitting (too general)!

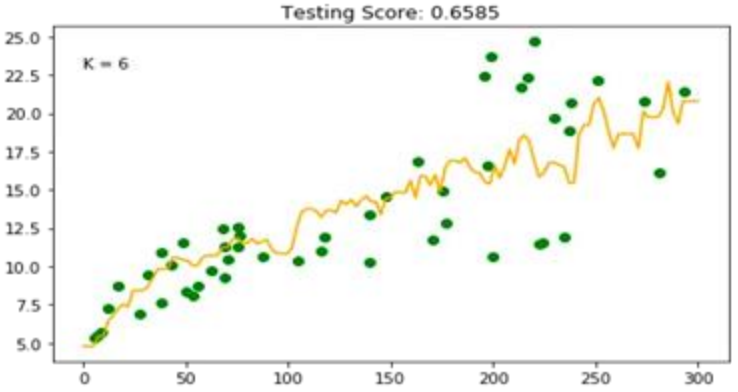
Why? consider all data points as neighbors, **overly generalized predictions** that might underfit the data by not effectively capturing local patterns.

- With k=n, the model uses all the training points for classification. This often leads to underfitting because it averages all points, missing local patterns
- Result in a simple decision boundary that might not capture the underlying patterns in the data effectively.
- simplifies the model too much
- often performs poorly because it's too simple to understand the data.

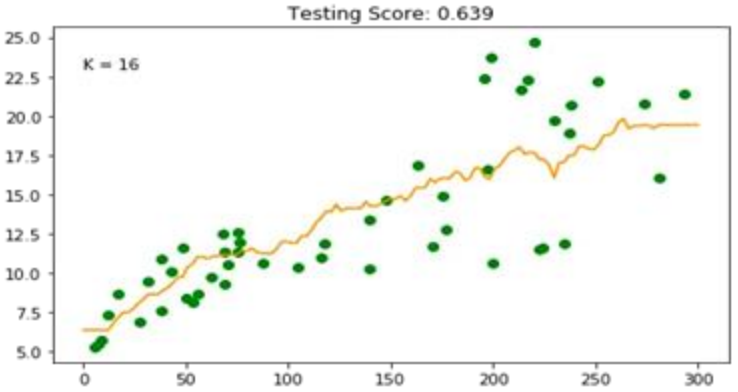


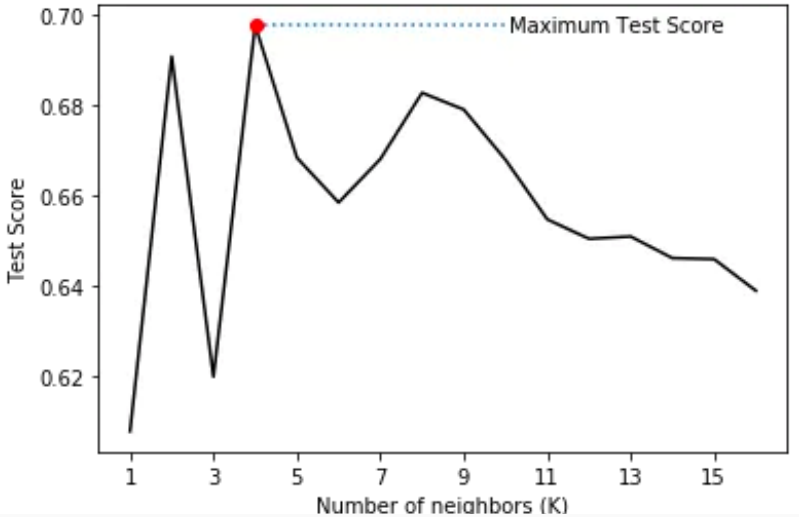












As K increases, the KNN fits a smoother curve to the data. Why? a higher value of K reduces the edginess by taking more data into account, thus reducing the overall complexity and flexibility of the model.

But observe top figure, **where increasing the value of K improves the score to a certain point, after which it again starts dropping (indicating underfitting).**

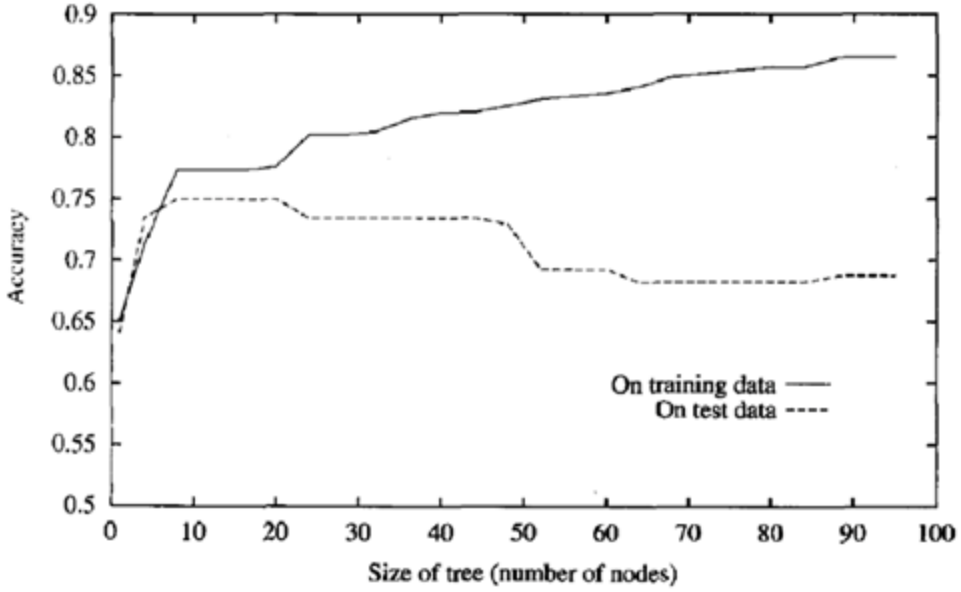
https://towardsdatascience.com/k-nearest-neighbors-94395f445221#:~:text=At%20K%3D1%2C%20the%20KNN,for%20different%20values%20of%20K.&text=As%20K%20increases%2C%20the%20KNN,smoother%20curve%20to%20the%20data.

Overfitting - How about decision tree?

Decision trees have a really bad overfitting problem!

A decision tree will use every scrap of information you give it and will memorize the entire training set

Decision-tree learners can create over-complex trees that do not generalize the data well. This is called overfitting.



How to deal with overfitting? Play with Hyperparameters

Hyperparameter: An argument that determines how the model works.

- K for KNN

Often times, we play around with a bunch of hyperparameters, and see what works best for the test set.

How to deal with overfitting in a decision tree

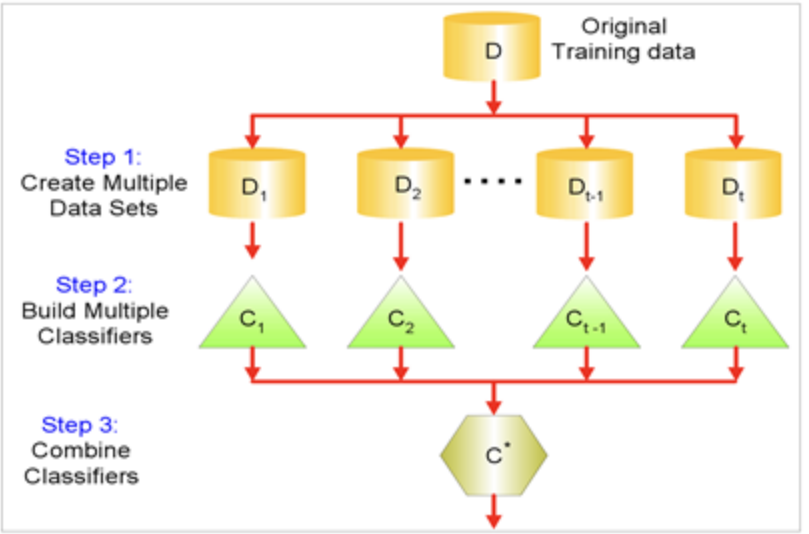
- Apply **a depth limit (fixed depth/ early stopping)**
- Apply **Pruning** where we just chop off everything after four levels
- We can try different ones and see how it affects our accuracy on the test set
- Or, use ensembles of different trees (random forests)

Ensemble methods

Ensemble methods, such as bagging and boosting, can also mitigate overfitting.

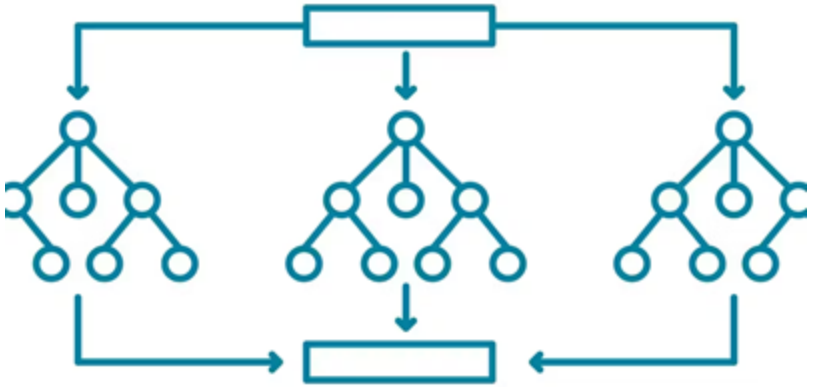
These techniques combine multiple models to make predictions, reducing the impact of individual model's biases and errors.

By leveraging the diversity of these models, ensemble methods enhance the generalization ability and minimize overfitting.



Random Forest

Random Forest is an ensemble learning method primarily used for classification and regression tasks.



- Builds a multitude of decision trees during training and merges their predictions to improve accuracy and control overfitting. Here’s how it works

Key Steps in Random Forest

Bootstrap Sampling (Bagging): creates N subsets of the original training data by randomly sampling with replacement. Each subset will be used to train a different decision tree.

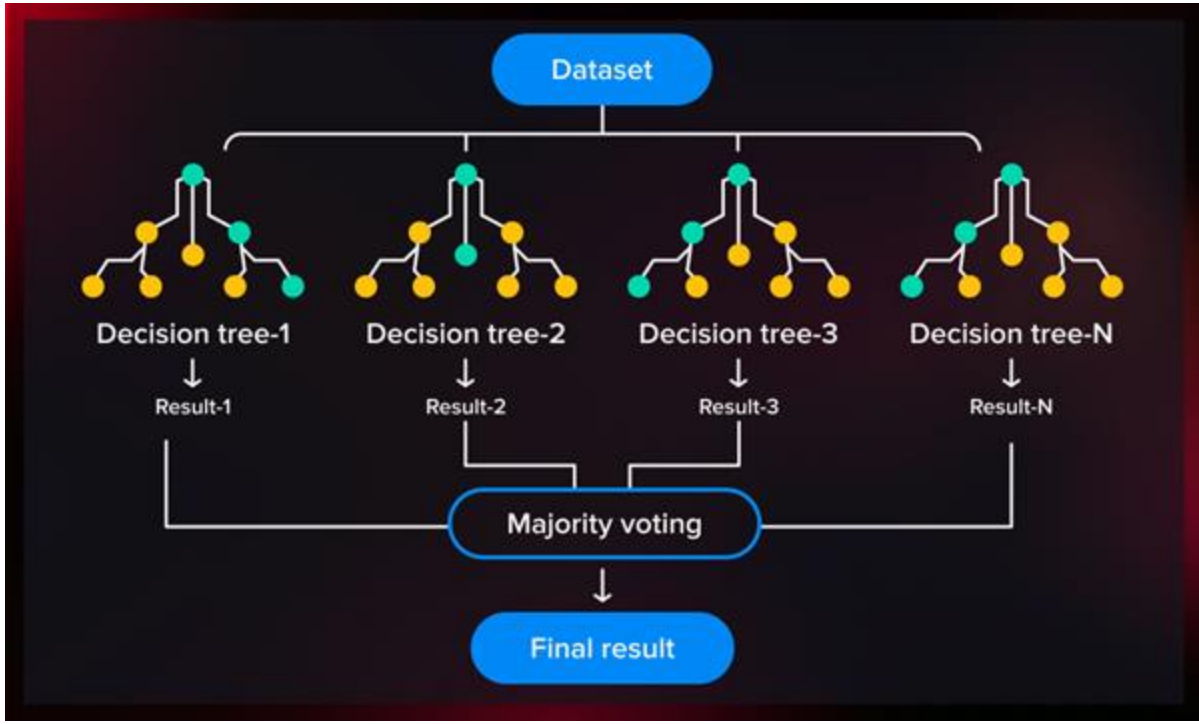
Building Decision Trees:

For each bootstrap sample, a decision tree is constructed. Random Forest randomly select features for splitting to increase tree diversity.

- Sampling with replacement

Training Data

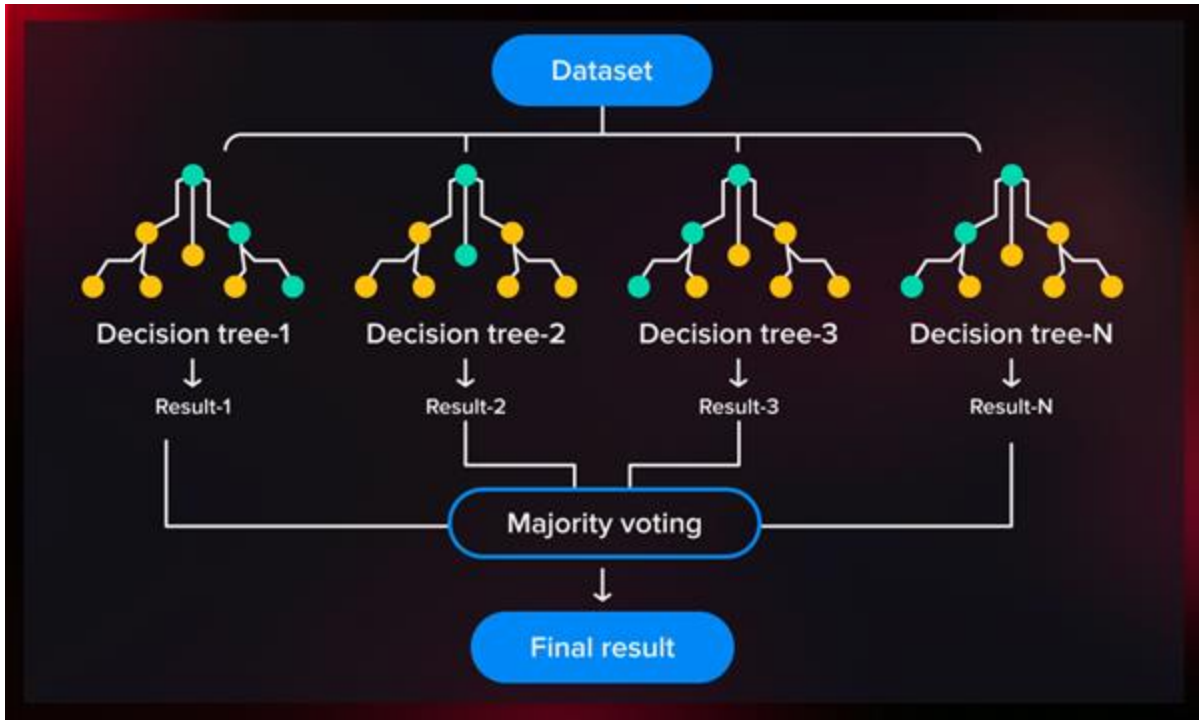
Data ID	1	2	3	4	5	6	7	8	9	10
Original Data										
Bagging (Round 1)	7	8	10	8	2	5	10	10	5	9
Bagging (Round 2)	1	4	9	1	2	3	2	7	3	2
Bagging (Round 3)	1	8	5	10	5	5	9	6	3	7



Key Steps in Random Forest

Predictions:

- **Classification:** Use majority voting from all trees.
- **Regression:** Calculate the average prediction from all trees.



❖ If a decision tree is overfit or has locked on to a weird correlation, it gets outvoted

❖ Not great for super high-dimensional data

'N' is a hyperparameter that determines the number of trees in the forest.

Usually the higher the number of trees the better to learn the data. However, adding a lot of trees can slow down the training process considerably, therefore we do a parameter search to find the sweet spot.

Random Forest Implementation using Scikit Learn

[16]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=44)
from sklearn.ensemble import RandomForestClassifier
rf_model = RandomForestClassifier(n_estimators=50, random_state=44)
rf_model.fit(X_train, y_train)
```

[16]:
```
▾ RandomForestClassifier
RandomForestClassifier(n_estimators=50, random_state=44)
```

How do Code?

Scikit Learn

So far, we have our data in a dataframe. Scikit learn has a classifier object that all classifiers inherit. It will have the following methods:

fit(X, y): A function that trains the model, where X is something like a dataframe of training data, and y is the labels

predict(X): Takes in unlabeled data and produces an array of labels

SKLearn Example

- **Load Iris dataset:** Imports the Iris dataset.
- **Initialize a Decision Tree Classifier:** Creates a decision tree classifier.
- **Train the Classifier:** Uses the classifier to fit (train) on the Iris dataset's features (iris.data) and their labels (iris.target).
- **Make Predictions:** Once trained, uses the trained model to predict labels for new, unlabeled data (unlabeled_data).

```
from sklearn.datasets import load_iris

from sklearn.tree import
DecisionTreeClassifier

clf = DecisionTreeClassifier()

clf.fit(iris.data, iris.target)

clf.predict(unlabeled_data)
```

SKLearn Example

```
from sklearn.datasets import load_iris

from sklearn.tree import DecisionTreeClassifier

clf = DecisionTreeClassifier(max_depth=4)

clf.fit(iris.data, iris.target)

clf.predict(unlabeled_data)
```

Controlling the depth can help prevent the model from overfitting and can lead to a more straightforward or less complex decision tree.

Decision Tree

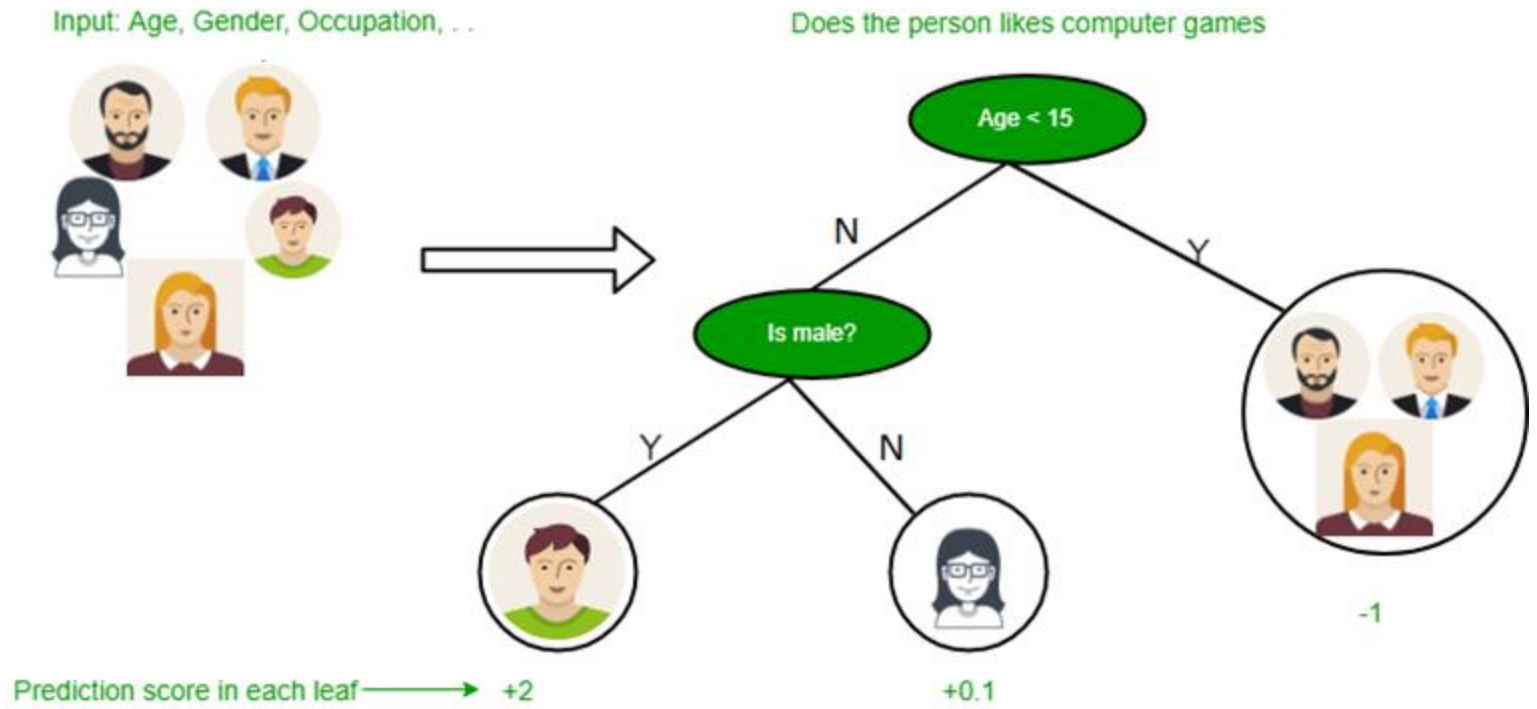
Topics:

- What is Decision Tree
- Common Terminology
- Concept of How to Split a Node
- Entropy
- Information Gain
- Overfitting Problem
  - Pre-Pruning
  - Post- Pruning

A Decision Tree

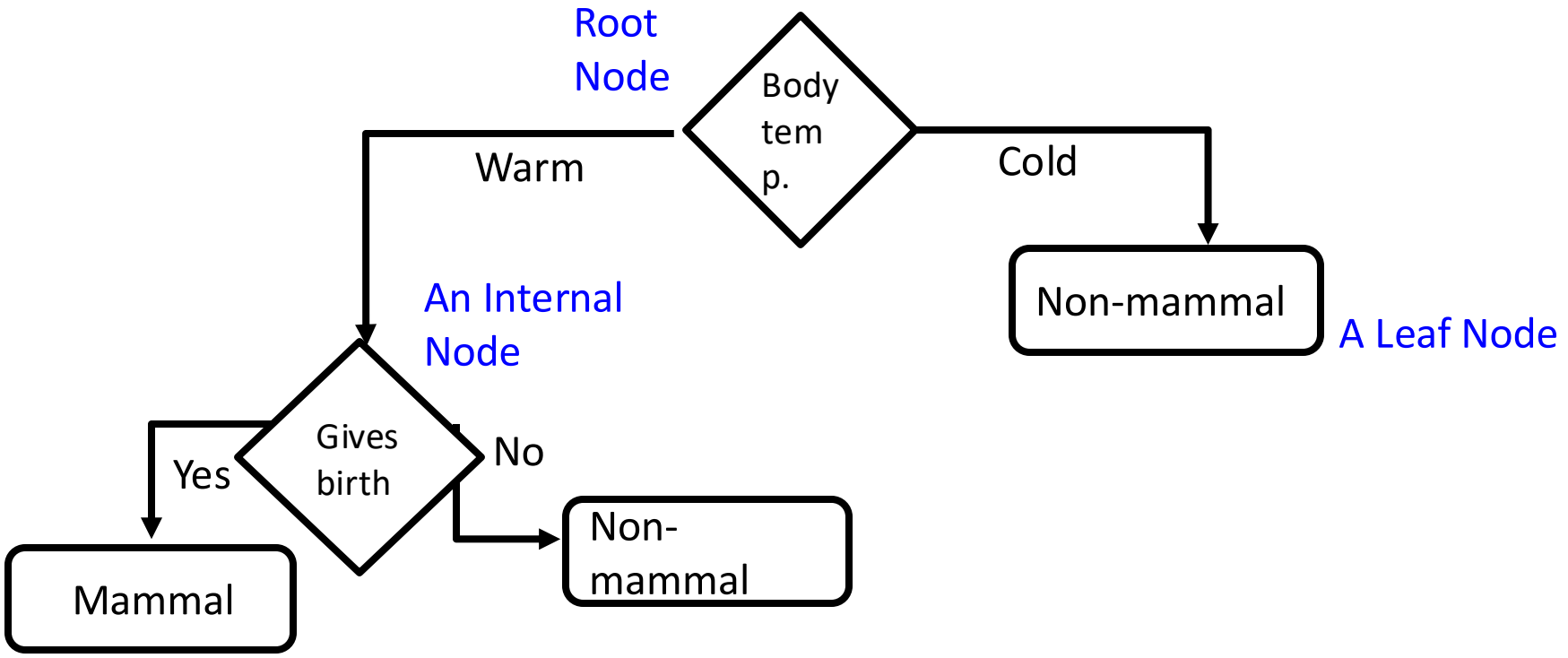
A decision tree is a popular machine learning algorithm **used for both classification and regression tasks**. It splits the dataset into subsets based on the most significant feature at each step.

Visually represents decisions and their possible consequences, making it easy to understand and interpret.



Decision Trees

- A Decision Tree (DT) defines a hierarchy of rules to make a prediction



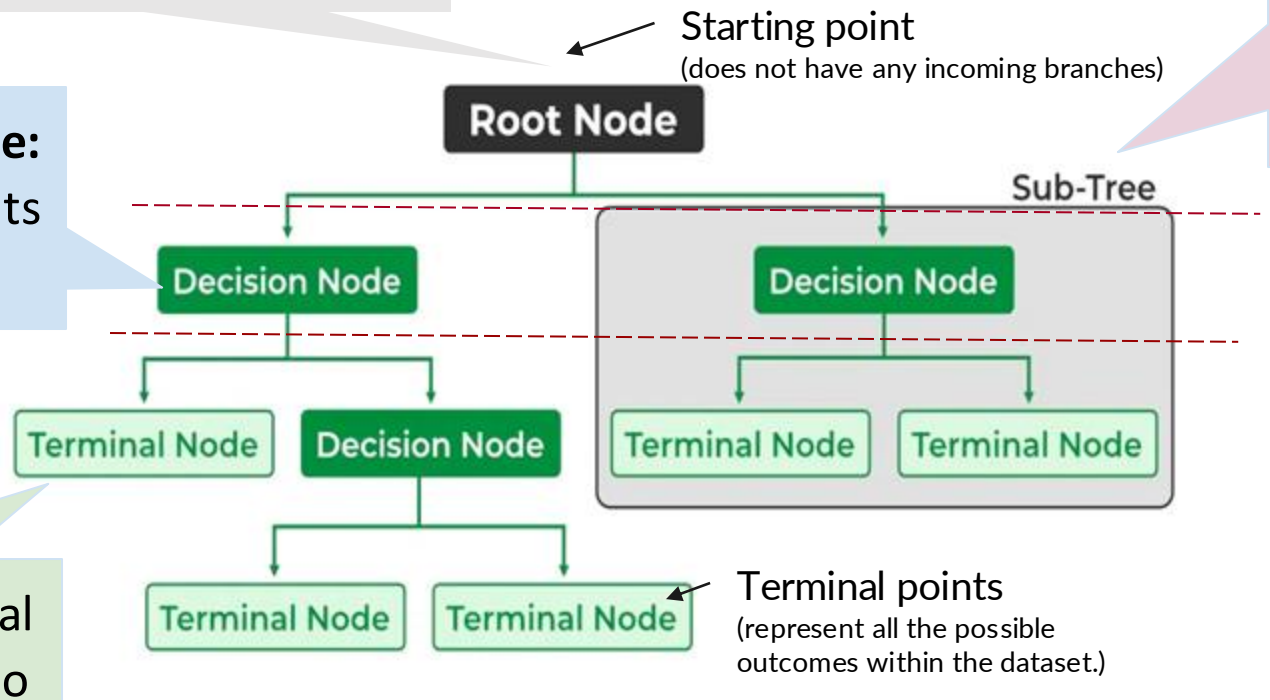
- Root and internal nodes test rules. Leaf nodes make predictions
- Decision Tree (DT) learning is about learning such a tree from labeled data

Terminology: Decision Tree Structure

Root Node: The top decision node (starting point for decision making).

Decision/Internal Node: Tests a feature and splits into branches.

Leaf/ Terminal Node: Final node with a prediction (no further splits).



Branch/Sub-Tree: A connection between nodes, representing a possible outcome based on a feature test.

Layer: Nodes at the same depth from the root.

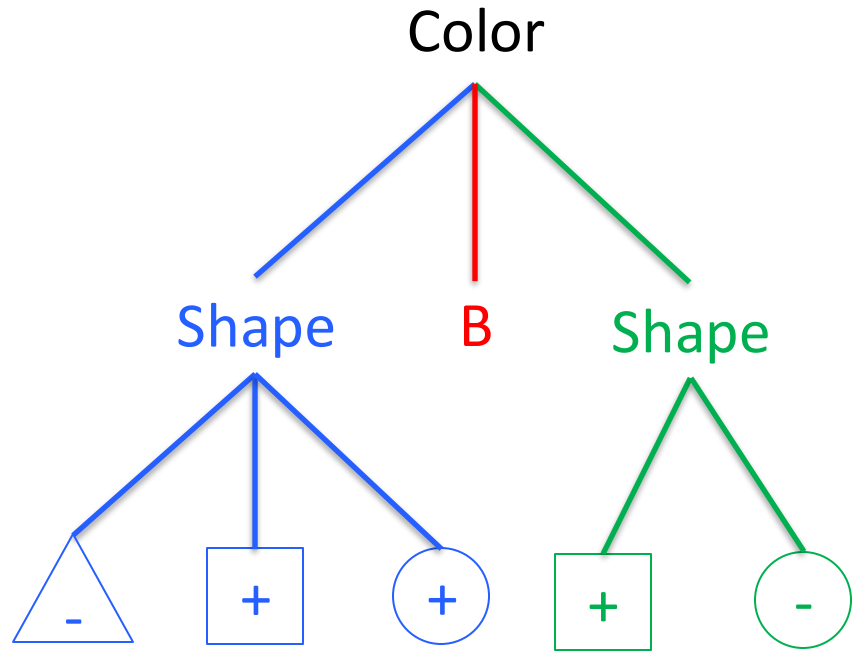
Depth: The maximum number of layers from root to farthest leaf.

A binary decision tree (BST) is a decision tree in which each node has two branches.

Output of Decision Trees

- **Classification Trees:** Output is a **discrete category** (e.g., "Yes/No", "Cat/Dog").
- **Regression trees:** Predict a continuous numerical value (e.g., house price, temperature).

Decision Rules: Each path from root to leaf represents an IF-THEN rule.

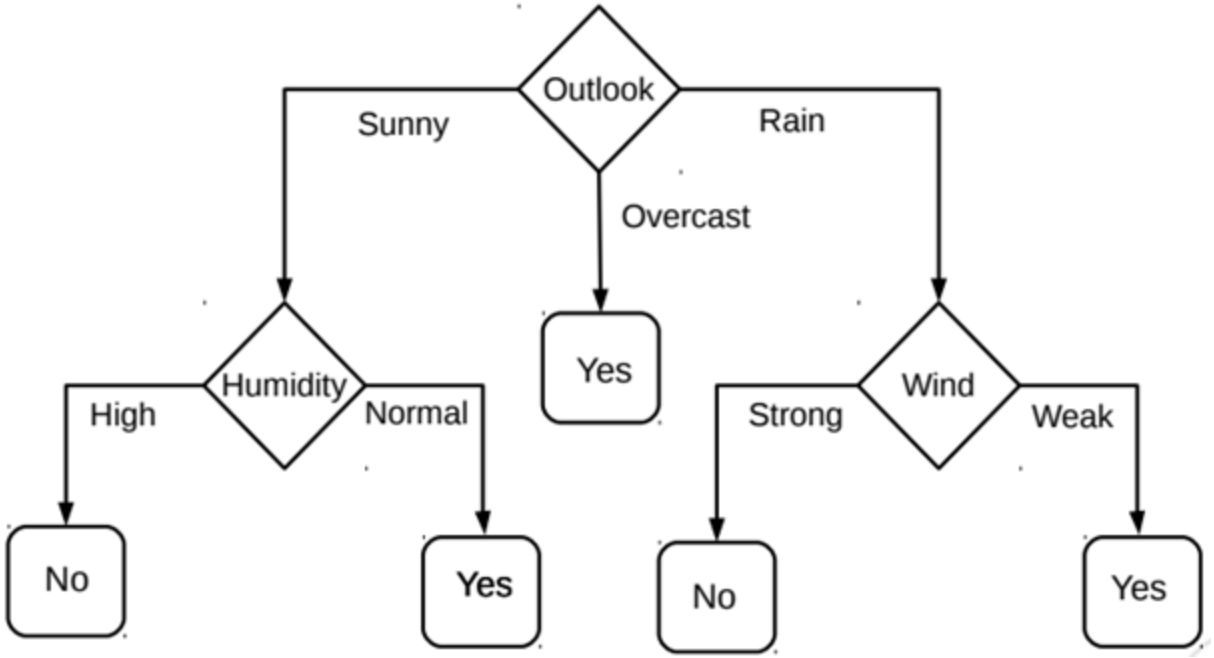


Decision Trees for Classification: Example

Predict: Deciding whether to play or not to play Tennis on a Saturday.

- Each input (Saturday) has 4 categorical features: Outlook, Temp., Humidity, Wind
- A binary classification problem (play vs no-play)
- **Below Left:** Training data, **Below Right:** A decision tree constructed using this data

day	outlook	temperature	humidity	wind	play
1	sunny	hot	high	weak	no
2	sunny	hot	high	strong	no
3	overcast	hot	high	weak	yes
4	rain	mild	high	weak	yes
5	rain	cool	normal	weak	yes
6	rain	cool	normal	strong	no
7	overcast	cool	normal	strong	yes
8	sunny	mild	high	weak	no
9	sunny	cool	normal	weak	yes
10	rain	mild	normal	weak	yes
11	sunny	mild	normal	strong	yes
12	overcast	mild	high	strong	yes
13	overcast	hot	normal	weak	yes
14	rain	mild	high	strong	no



Predict if John will play tennis

Training examples: **9 yes / 5 no**

Day	Outlook	Humidity	Wind	Play
D1	Sunny	High	Weak	No
D2	Sunny	High	Strong	No
D3	Overcast	High	Weak	Yes
D4	Rain	High	Weak	Yes
D5	Rain	Normal	Weak	Yes
D6	Rain	Normal	Strong	No
D7	Overcast	Normal	Strong	Yes
D8	Sunny	High	Weak	No
D9	Sunny	Normal	Weak	Yes
D10	Rain	Normal	Weak	Yes
D11	Sunny	Normal	Strong	Yes
D12	Overcast	High	Strong	Yes
D13	Overcast	Normal	Weak	Yes
D14	Rain	High	Strong	No

New data:

D15	Rain	High	Weak	?

Predict if John will play tennis

- HARD TO GUESS!
- TRY TO UNDERSTAND WHEN JOHN PLAY.

Solution: Decision Tree

- Divide & conquer:
  - split into subsets
  - are they pure? (all yes or all no)
  - if yes: stop
  - if not: repeat
- See which subset new data falls into

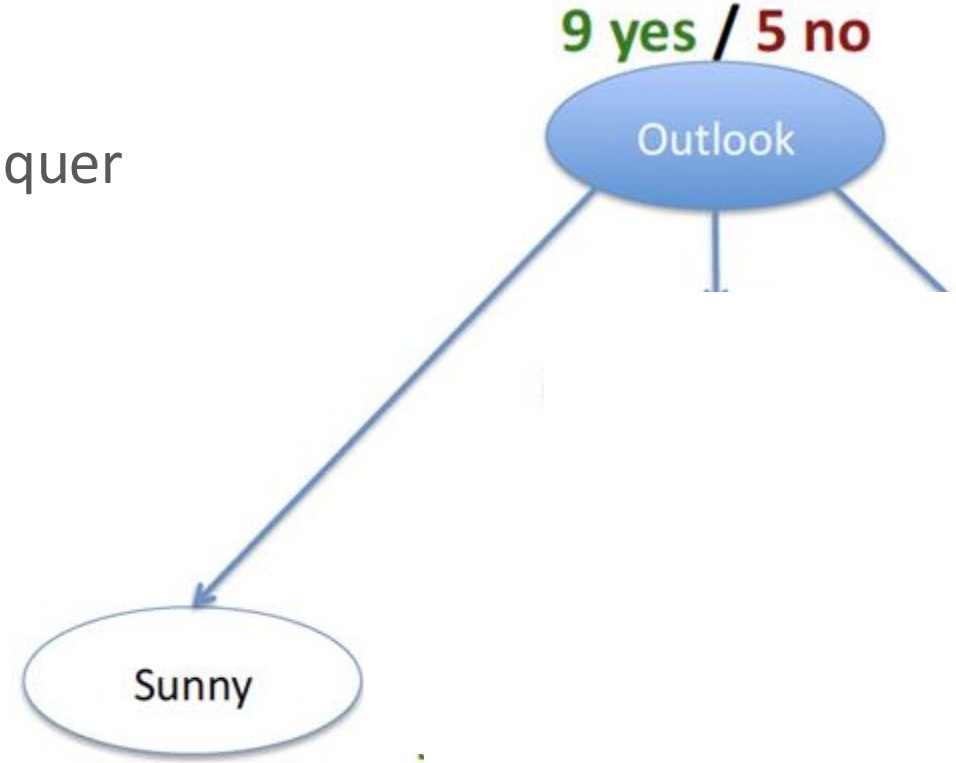
Training examples: **9 yes / 5 no**

Day	Outlook	Humidity	Wind	Play
D1	Sunny	High	Weak	No
D2	Sunny	High	Strong	No
D3	Overcast	High	Weak	Yes
D4	Rain	High	Weak	Yes
D5	Rain	Normal	Weak	Yes
D6	Rain	Normal	Strong	No
D7	Overcast	Normal	Strong	Yes
D8	Sunny	High	Weak	No
D9	Sunny	Normal	Weak	Yes
D10	Rain	Normal	Weak	Yes
D11	Sunny	Normal	Strong	Yes
D12	Overcast	High	Strong	Yes
D13	Overcast	Normal	Weak	Yes
D14	Rain	High	Strong	No

New data:

D15	Rain	High	Weak	?

Decision Tree:
Divide and Conquer



Day	Outlook	Humid	Wind
D1	Sunny	High	Weak
D2	Sunny	High	Strong
D8	Sunny	High	Weak
D9	Sunny	Normal	Weak
D11	Sunny	Normal	Strong

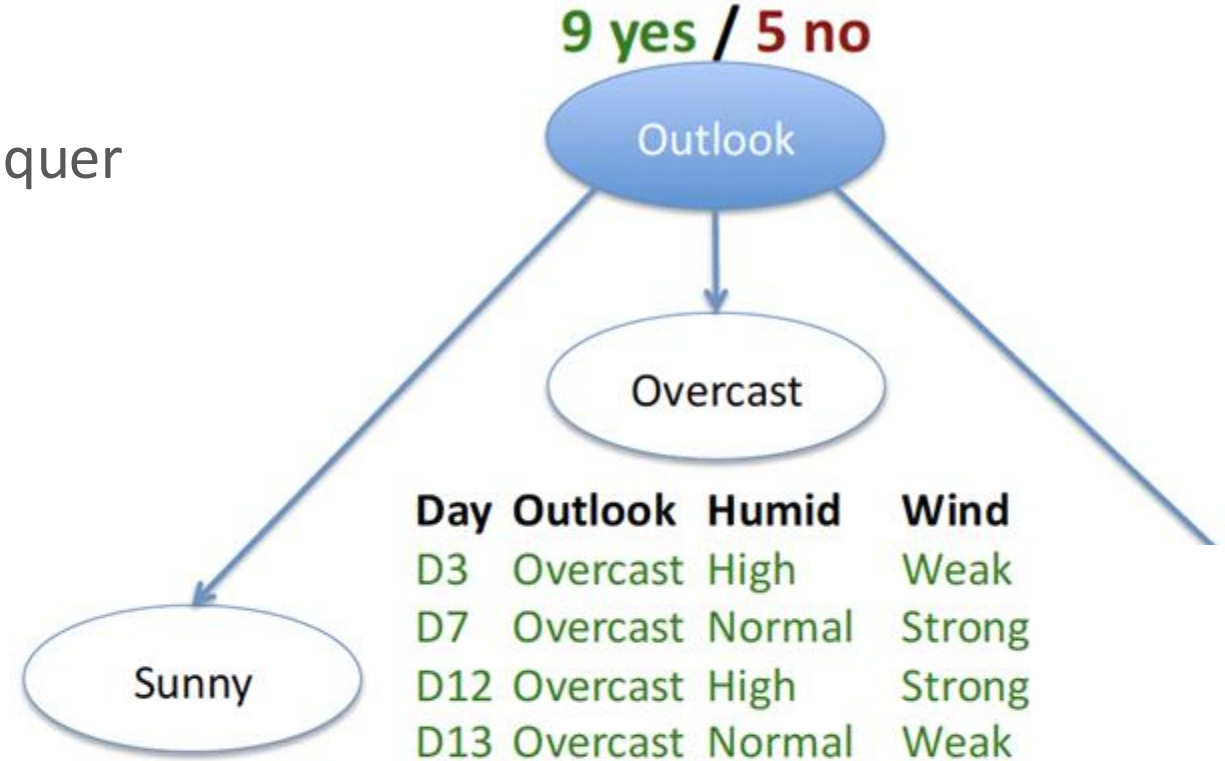
2 yes / 3 no
split further

Training examples: **9 yes / 5 no**

Day	Outlook	Humidity	Wind	Play
D1	Sunny	High	Weak	No
D2	Sunny	High	Strong	No
D3	Overcast	High	Weak	Yes
D4	Rain	High	Weak	Yes
D5	Rain	Normal	Weak	Yes
D6	Rain	Normal	Strong	No
D7	Overcast	Normal	Strong	Yes
D8	Sunny	High	Weak	No
D9	Sunny	Normal	Weak	Yes
D10	Rain	Normal	Weak	Yes
D11	Sunny	Normal	Strong	Yes
D12	Overcast	High	Strong	Yes
D13	Overcast	Normal	Weak	Yes
D14	Rain	High	Strong	No

- Divide & conquer:
  - split into subsets
  - are they pure? (all yes or all no)
  - if yes: stop
  - if not: repeat
- See which subset new data falls into

Decision Tree:
Divide and Conquer



Day	Outlook	Humid	Wind
D3	Overcast	High	Weak
D7	Overcast	Normal	Strong
D12	Overcast	High	Strong
D13	Overcast	Normal	Weak

4 yes / 0 no
pure subset

Day	Outlook	Humid	Wind
D1	Sunny	High	Weak
D2	Sunny	High	Strong
D8	Sunny	High	Weak
D9	Sunny	Normal	Weak
D11	Sunny	Normal	Strong

2 yes / 3 no
split further

Training examples: 9 yes / 5 no

Day	Outlook	Humidity	Wind	Play
D1	Sunny	High	Weak	No
D2	Sunny	High	Strong	No
D3	Overcast	High	Weak	Yes
D4	Rain	High	Weak	Yes
D5	Rain	Normal	Weak	Yes
D6	Rain	Normal	Strong	No
D7	Overcast	Normal	Strong	Yes
D8	Sunny	High	Weak	No
D9	Sunny	Normal	Weak	Yes
D10	Rain	Normal	Weak	Yes
D11	Sunny	Normal	Strong	Yes
D12	Overcast	High	Strong	Yes
D13	Overcast	Normal	Weak	Yes
D14	Rain	High	Strong	No

- Divide & conquer:
  - split into subsets
  - are they pure? (all yes or all no)
  - if yes: stop
  - if not: repeat
- See which subset new data falls into

Training examples: **9 yes / 5 no**

Day	Outlook	Humidity	Wind	Play
D1	Sunny	High	Weak	No
D2	Sunny	High	Strong	No
D3	Overcast	High	Weak	Yes
D4	Rain	High	Weak	Yes
D5	Rain	Normal	Weak	Yes
D6	Rain	Normal	Strong	No
D7	Overcast	Normal	Strong	Yes
D8	Sunny	High	Weak	No
D9	Sunny	Normal	Weak	Yes
D10	Rain	Normal	Weak	Yes
D11	Sunny	Normal	Strong	Yes
D12	Overcast	High	Strong	Yes
D13	Overcast	Normal	Weak	Yes
D14	Rain	High	Strong	No



9 yes / 5 no









Day	Outlook	Humid	Wind
D3	Overcast	High	Weak
D7	Overcast	Normal	Strong
D12	Overcast	High	Strong
D13	Overcast	Normal	Weak

4 yes / 0 no

pure subset

Day	Outlook	Humid	Wind
D1	Sunny	High	Weak
D2	Sunny	High	Strong
D8	Sunny	High	Weak
D9	Sunny	Normal	Weak
D11	Sunny	Normal	Strong

2 yes / 3 no

split further

Day	Outlook	Humid	Wind
D4	Rain	High	Weak
D5	Rain	Normal	Weak
D6	Rain	Normal	Strong
D10	Rain	Normal	Weak
D14	Rain	High	Strong

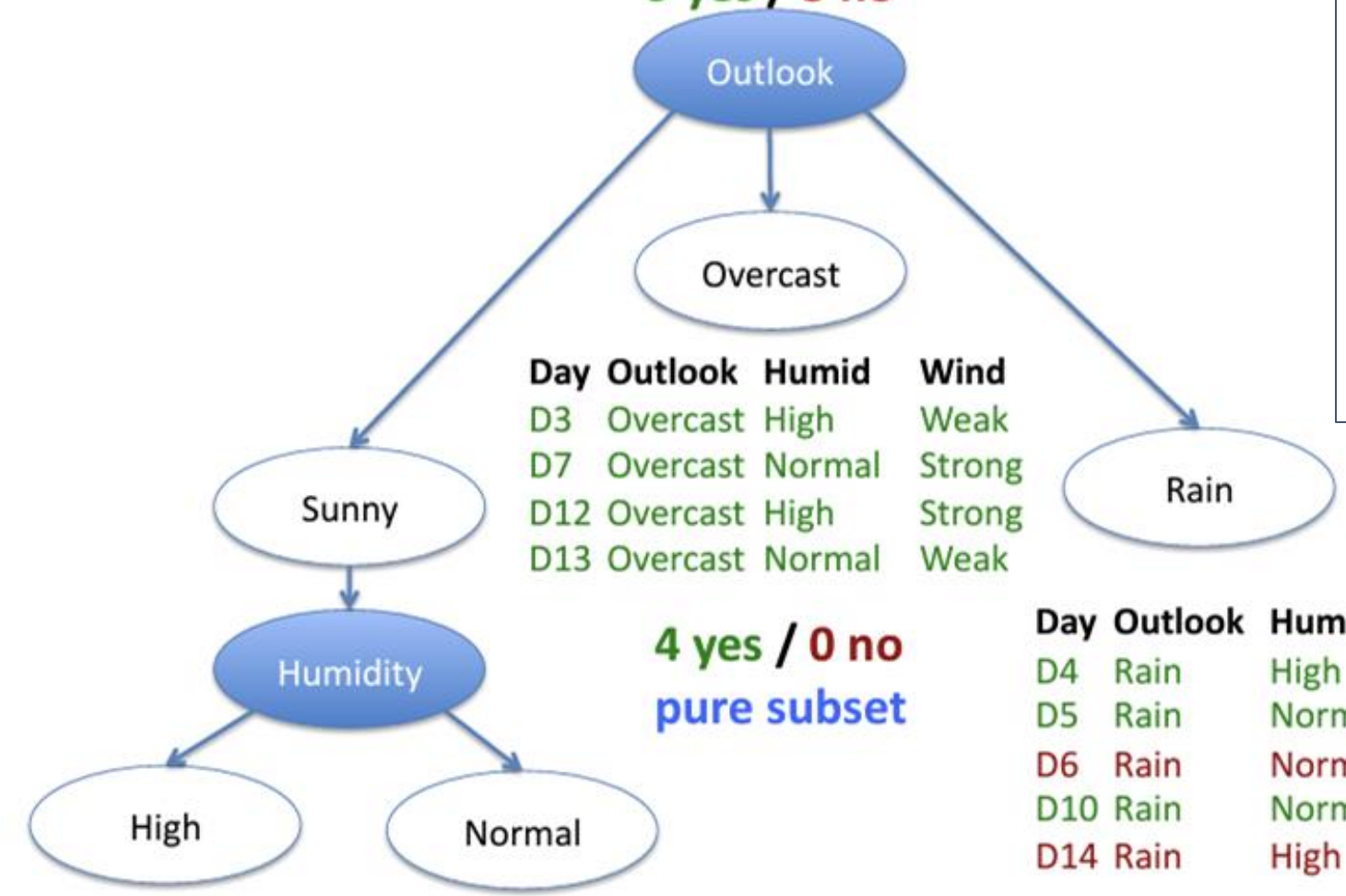
3 yes / 2 no

split further

We need to figure out a way to split further in order to get pure subsets.

Training examples: **9 yes / 5 no**

Day	Outlook	Humidity	Wind	Play
D1	Sunny	High	Weak	No
D2	Sunny	High	Strong	No
D3	Overcast	High	Weak	Yes
D4	Rain	High	Weak	Yes
D5	Rain	Normal	Weak	Yes
D6	Rain	Normal	Strong	No
D7	Overcast	Normal	Strong	Yes
D8	Sunny	High	Weak	No
D9	Sunny	Normal	Weak	Yes
D10	Rain	Normal	Weak	Yes
D11	Sunny	Normal	Strong	Yes
D12	Overcast	High	Strong	Yes
D13	Overcast	Normal	Weak	Yes
D14	Rain	High	Strong	No



Day	Outlook	Humid	Wind
D3	Overcast	High	Weak
D7	Overcast	Normal	Strong
D12	Overcast	High	Strong
D13	Overcast	Normal	Weak

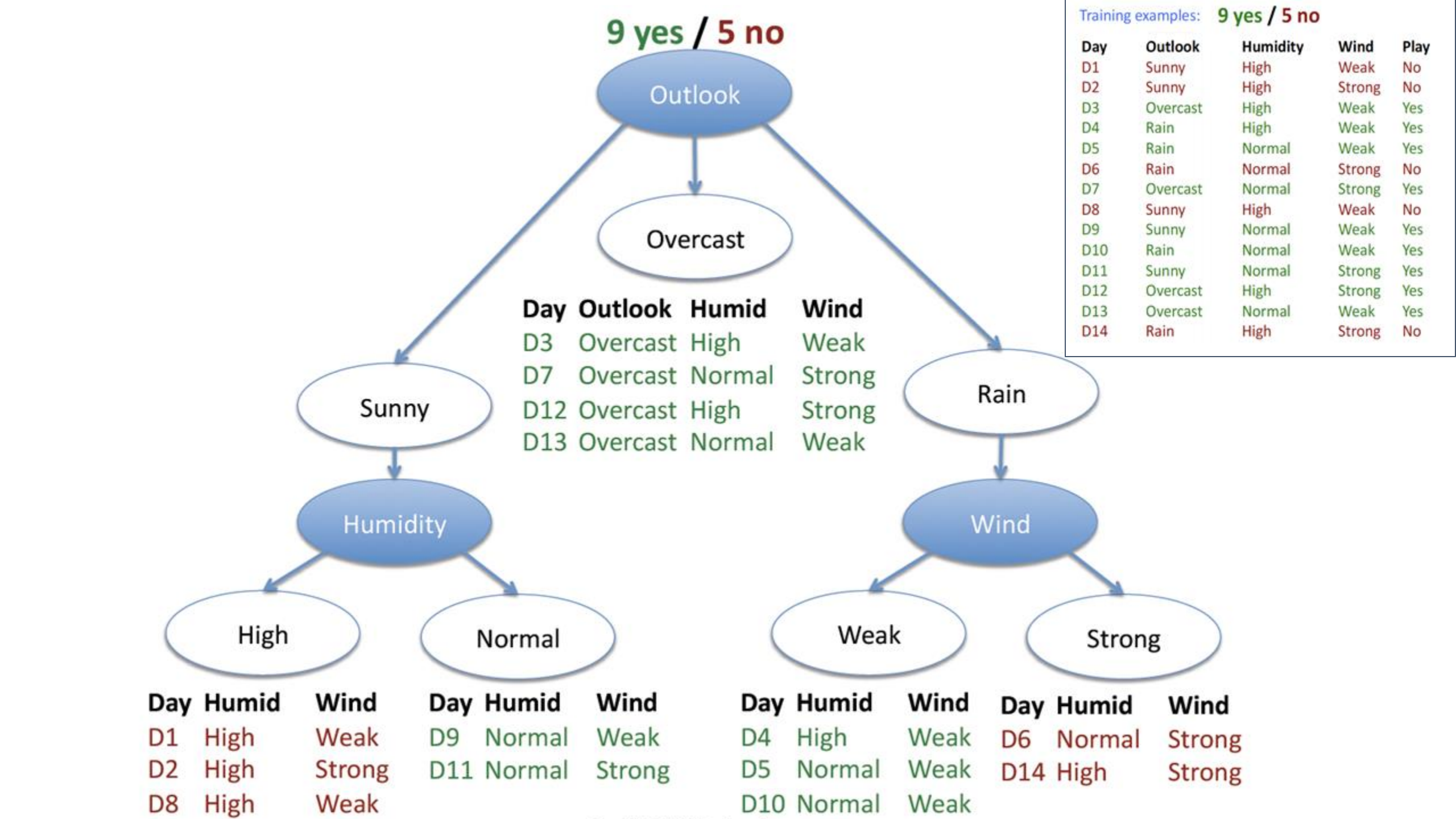
4 yes / 0 no
pure subset

Day	Outlook	Humid	Wind
D4	Rain	High	Weak
D5	Rain	Normal	Weak
D6	Rain	Normal	Strong
D10	Rain	Normal	Weak
D14	Rain	High	Strong

3 yes / 2 no
split further

Day	Humid	Wind
D1	High	Weak
D2	High	Strong
D8	High	Weak

Day	Humid	Wind
D9	Normal	Weak
D11	Normal	Strong

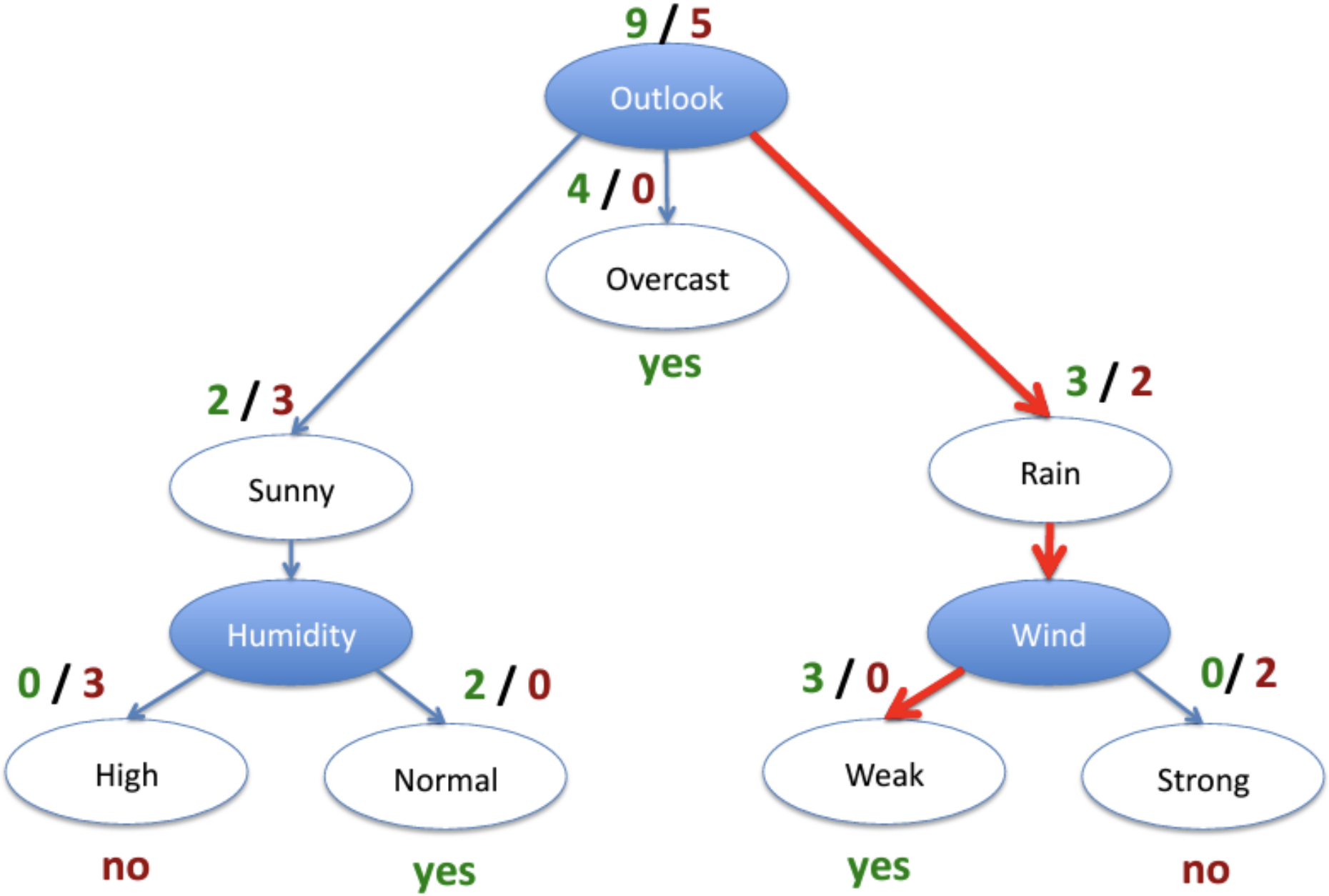


Training examples: **9 yes / 5 no**

Day	Outlook	Humidity	Wind	Play
D1	Sunny	High	Weak	No
D2	Sunny	High	Strong	No
D3	Overcast	High	Weak	Yes
D4	Rain	High	Weak	Yes
D5	Rain	Normal	Weak	Yes
D6	Rain	Normal	Strong	No
D7	Overcast	Normal	Strong	Yes
D8	Sunny	High	Weak	No
D9	Sunny	Normal	Weak	Yes
D10	Rain	Normal	Weak	Yes
D11	Sunny	Normal	Strong	Yes
D12	Overcast	High	Strong	Yes
D13	Overcast	Normal	Weak	Yes
D14	Rain	High	Strong	No

Resulting Final Decision Tree

A logical Predicates or formula that told you in which cases John will play and vice versa.



New data:

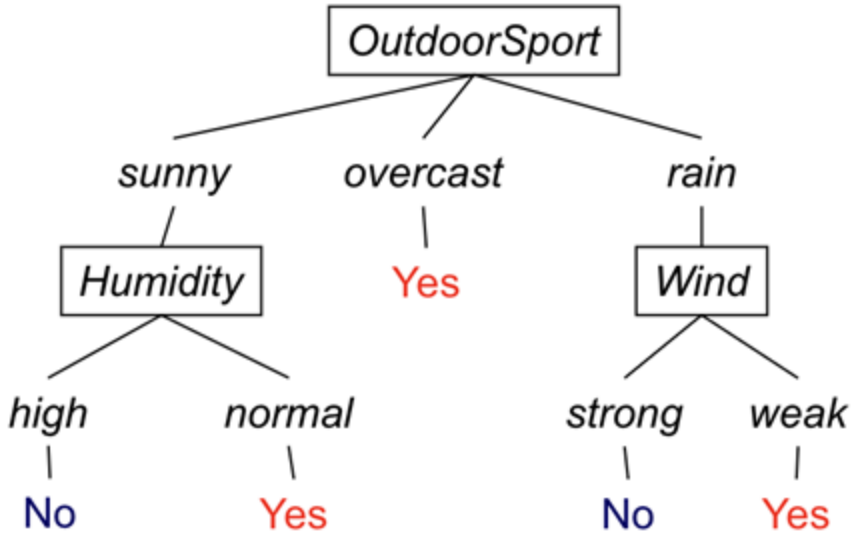
Day	Outlook	Humid	Wind	
D15	Rain	High	Weak	→ Yes

Concept Learning in Decision Trees

Decision trees learn boolean functions (True/False outputs) from examples.

Representation: Use a disjunction (OR) of conjunctions (AND) of attributes.

- Each path = conjunction (AND) of conditions
- Full tree = disjunction (OR) of all paths
  - **Eg.** IF (Outlook=Sunny AND Humidity=High) OR IF (Outlook= Rain AND Wind=Strong) THEN Play Tennis=No



Expressivity of Decision Trees:

- Can represent any boolean function (AND, OR, XOR).
- But may require exponentially large trees for some functions (e.g., XOR).
- Human-interpretable decision rules
- The disjunction of these rules is equivalent to the Decision Tree

Design Principle:

- Prefer simpler trees (**Occam's Razor:** simpler hypotheses generalize better).
- Fewer splits → better generalization

Decision Tree Induction

The process of automatically constructing a decision tree from training data

Many Algorithms:

- ID3 (Iterative Dichotomiser 3)
- C4.5
- Hunt's Algorithm
- CART
- SLIQ,SPRINT

ID3 algorithm

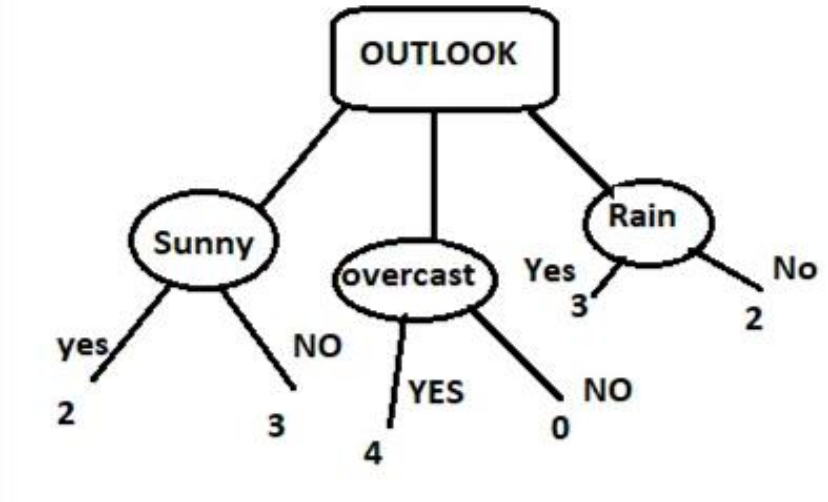
- Split (node, {examples}):
  1. A ← the best attribute for splitting the {examples}
  2. Decision attribute for this node ← A
  3. For each value of A, create new child node
  4. Split training {examples} to child nodes
  5. If examples perfectly classified: STOP
     else: iterate over new child nodes
     Split (child_node, {subset of examples})
- Ross Quinlan (ID3: 1986), (C4.5: 1993)
- Breimanetal (CaRT: 1984) from statistics

Key Concepts: Entropy and Information Gain

Decision trees use entropy and information gain to decide the best feature for splitting.

Concept of “pure node”

A node where **all (or majority)** samples or data points belong to the same class (no further splitting needed). Thus it is very easy to make the prediction at such node.



Here, “Overcast” is a pure node

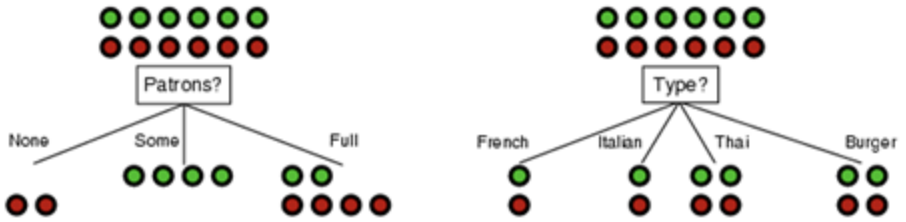
Impure Node: Requires splitting to separate mixed classes. Here “Sunny”, “Rain” are impure node → need splitting

Concept of “pure node”

Purity Goal: Seek splits that create subsets that are: Either all positive (e.g., 4Y/0N) Or all negative (e.g., 0Y/4N)

Try:

Idea: a good attribute splits the examples into subsets that are (ideally) “all positive” or “all negative”



Which split is more informative: *Patrons?* or *Type?*

Which attribute to split on?



- Want to measure “purity” of the split
  - more certain about Yes/No after the split
    - pure set (4 yes / 0 no) => completely certain (100%)
    - impure (3 yes / 3 no) => completely uncertain (50%)
  - can’t use P(“yes” | set):
    - must be symmetric: 4 yes / 0 no as pure as 0 yes / 4 no

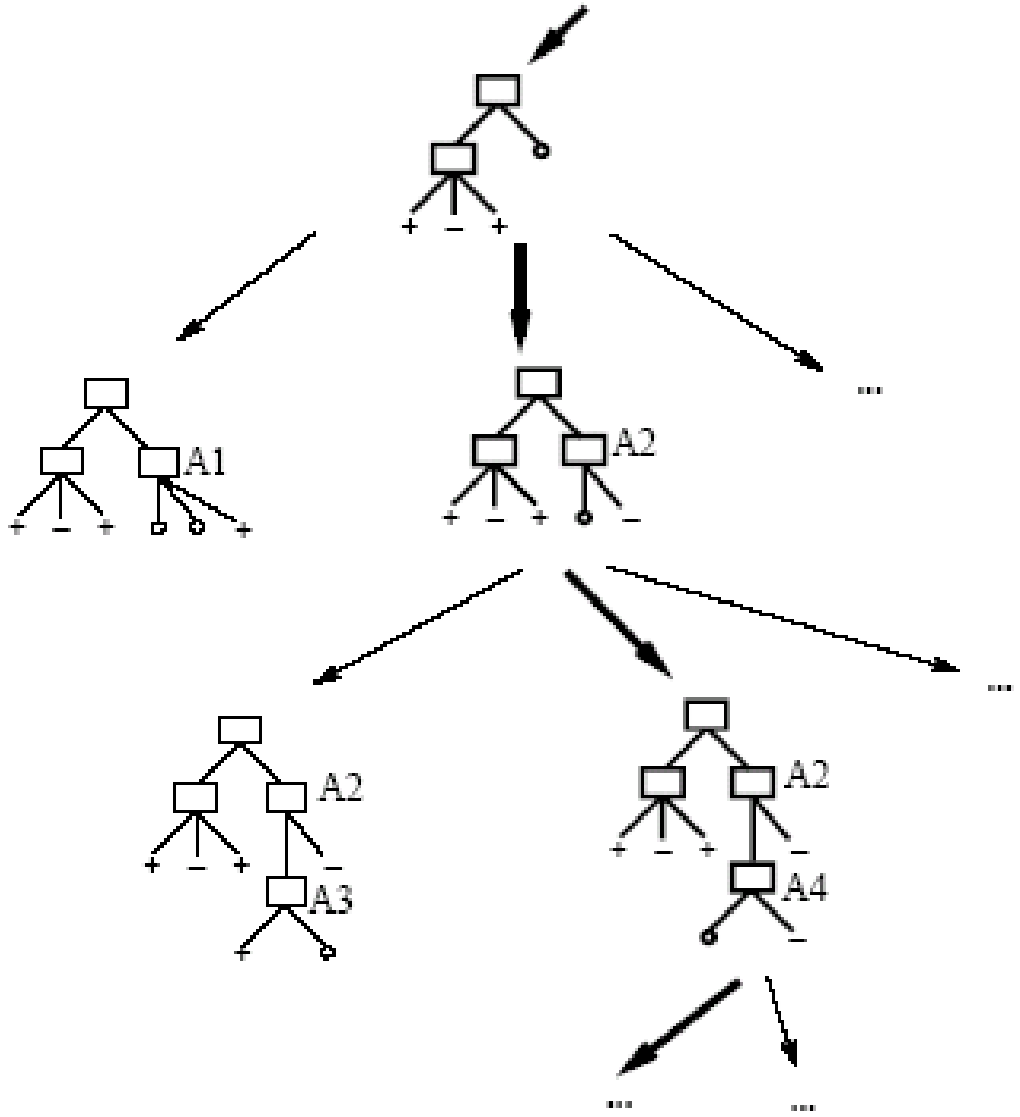
Hypothesis Space & Learning Challenges

Multiple trees can fit the same data →

Many possible hypotheses

Challenge: Learn the best tree from data

- Minimal tree search is NP-hard → use rule of thumb / educated guess
- Greedy algorithms are practical but suboptimal
- Split selection is the core computational challenge



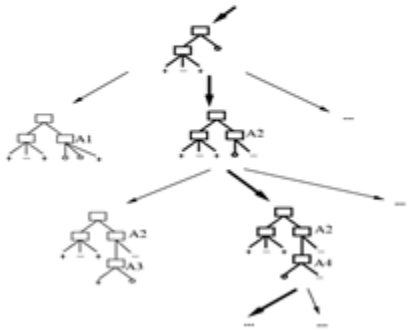
Remember: Ockham's Razor

The goal is to have the resulting decision tree as small as possible (Occam's Razor)

- The main decision in the algorithm is the selection of the next attribute

- Therefore, the smallest decision tree that correctly classifies all of the training examples is best
  - Finding the provably smallest decision tree is NP-hard
  - ...So instead of constructing the absolute smallest tree consistent with the training examples, construct one that is pretty small

Which Tree Should We Output?



- ID3 performs heuristic search through space of decision trees
- It stops at smallest acceptable tree. Why?

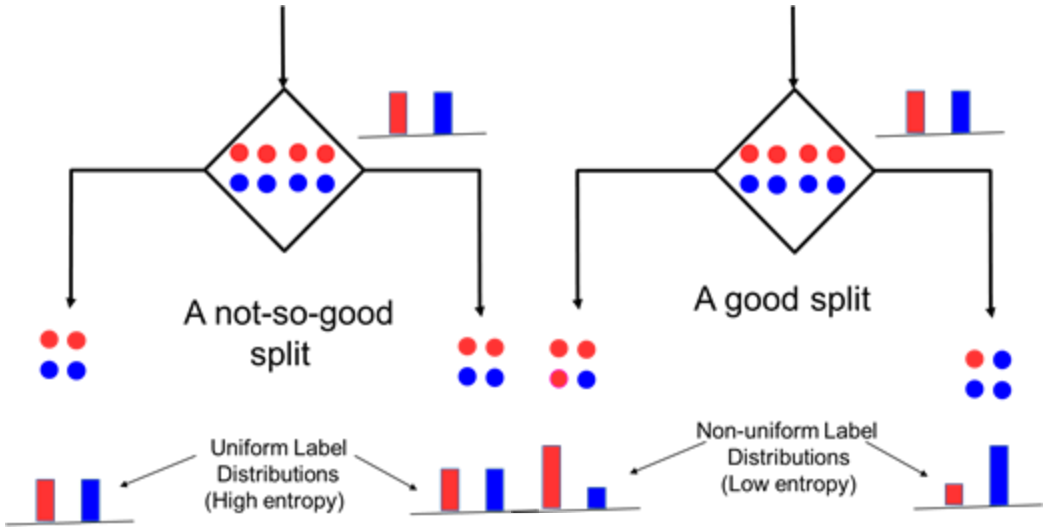
Occam's razor: prefer the simplest hypothesis that fits the data

Back to: How to Split at Internal Nodes?

Decision Tree Splitting: Purity & Entropy

Goal: Create maximally pure child nodes

- Pure = Majority of samples share same class



- For classification problems **(discrete outputs)**, entropy measures "purity".
  - **Low entropy (e.g., 0.0)** ⇒ high purity (less uniform label distribution → labels are mostly the same)
  - **High entropy (e.g., 1.0)** ⇒ Low purity (uniform label distribution → labels are mix)

Optimal Split: Maximizes information gain: Entropy(before) – Weighted Entropy(after)

- Key Points: Pure nodes stop further splitting; Greedily select highest IG at each step

– Measures the level of **impurity** in a group of examples

Entropy (Measure of Impurity)



Measure of **uncertainty or impurity in a dataset**. In the context of classification, it quantifies how mixed the class labels are.

If a dataset S contains c classes, the entropy of S is:

$$Entropy(S) = -\sum_{i=1}^{c} p_i \log_2(p_i)$$

Where:

- p_i = proportion of class i in the dataset.
- The log base 2 reflects the measurement in bits.

Interpretation:

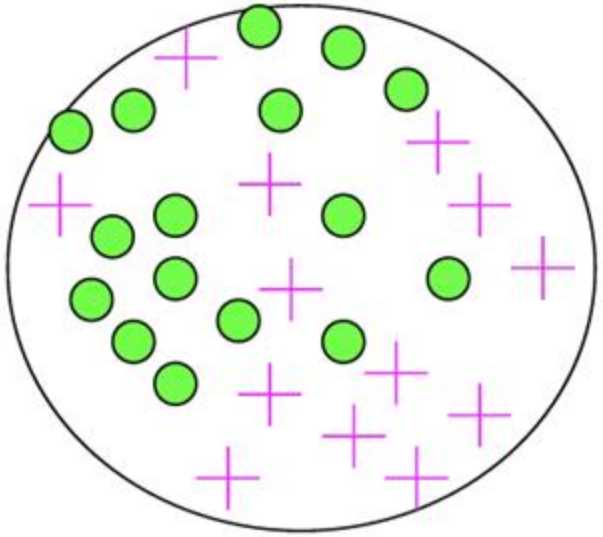
- **Entropy = 0** → Pure (all same class).
- **Entropy = 1** → Maximum impurity (equal class distribution).

Example:

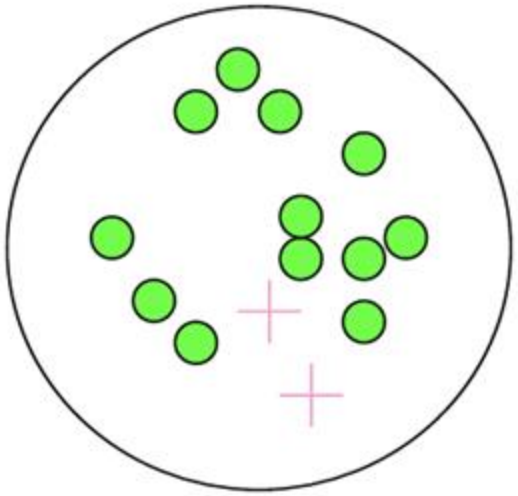
- 5 "Yes", 5 "No" → Entropy = 1.
- 8 "Yes", 2 "No" → Entropy ≈ 0.72.

Example: Impurity/ Entropy

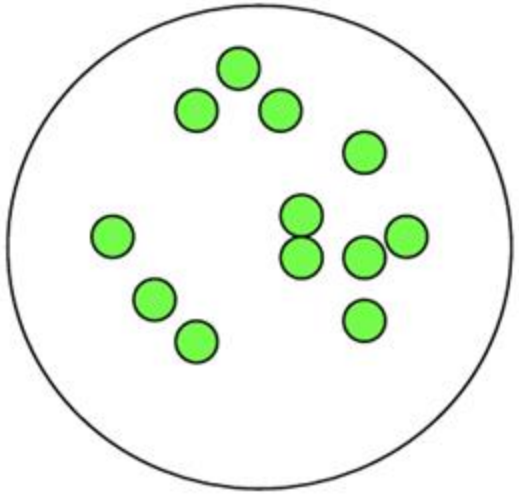
Very impure group



Less impure

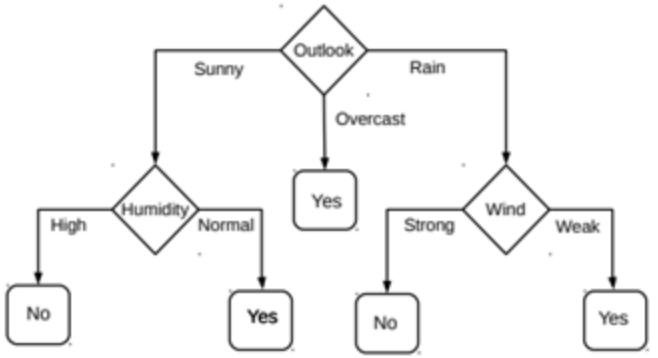


Minimum impurity

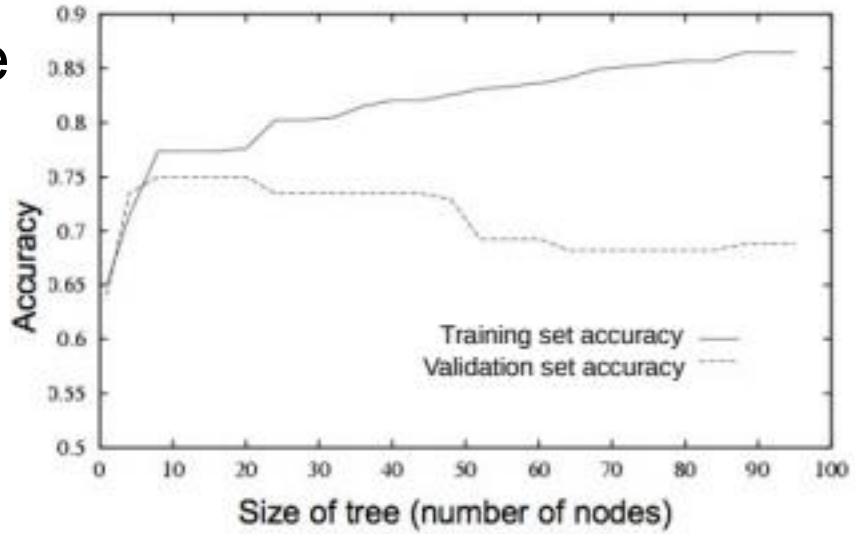


When to stop growing the tree?

day	outlook	temperature	humidity	wind	play
1	sunny	hot	high	weak	no
2	sunny	hot	high	strong	no
3	overcast	hot	high	weak	yes
4	rain	mild	high	weak	yes
5	rain	cool	normal	weak	yes
6	rain	cool	normal	strong	no
7	overcast	cool	normal	strong	yes
8	sunny	mild	high	weak	no
9	sunny	cool	normal	weak	yes
10	rain	mild	normal	weak	yes
11	sunny	mild	normal	strong	yes
12	overcast	mild	high	strong	yes
13	overcast	hot	normal	weak	yes
14	rain	mild	high	strong	no

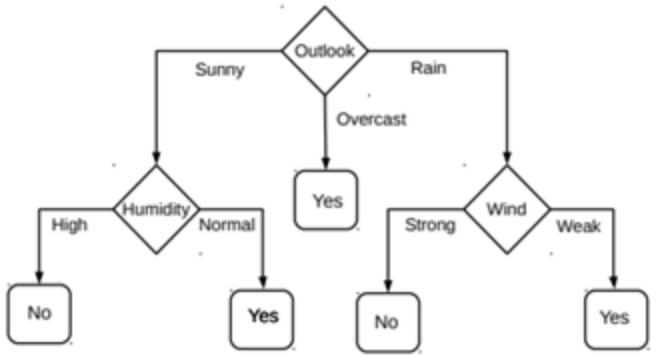


- Stop expanding a node further (i.e., make it a leaf node) when:
  - It consist of all training examples having the same label (the node becomes “pure”)
  - We run out of features to test along the path to that node
  - The DT starts to overfit (can be checked by monitoring the validation set accuracy)



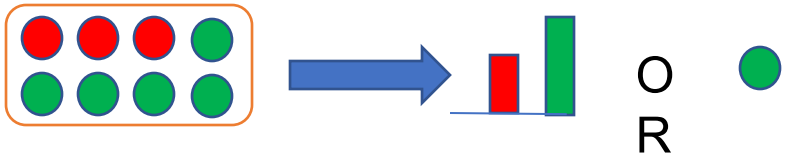
When to stop growing the tree?

day	outlook	temperature	humidity	wind	play
1	sunny	hot	high	weak	no
2	sunny	hot	high	strong	no
3	overcast	hot	high	weak	yes
4	rain	mild	high	weak	yes
5	rain	cool	normal	weak	yes
6	rain	cool	normal	strong	no
7	overcast	cool	normal	strong	yes
8	sunny	mild	high	weak	no
9	sunny	cool	normal	weak	yes
10	rain	mild	normal	weak	yes
11	sunny	mild	normal	strong	yes
12	overcast	mild	high	strong	yes
13	overcast	hot	normal	weak	yes
14	rain	mild	high	strong	no



- **Important:** No need to obsess too much for purity

(To help prevent the tree from growing too much!)



  - It is okay to have a leaf node that is not fully pure, e.g., this
  - **Predictions at Impure Leaf Nodes:** For test inputs that reach an impure leaf, we can predict probability of belonging to each class (in above example, p(red) = 3/8, p(green) = 5/8), or simply predict the majority label, which in this case would be green.

Avoiding Overfitting in DTs

- **Desired:** a DT that is not too big in size, yet fits the training data reasonably

- **Mainly two approaches to prune a complex DT**
  - Prune while building the tree (stopping early)
  - Prune after building the tree (post-pruning)

Avoid overfitting

- Stop splitting when not statistically significant
- Grow, then post-prune
  - based on validation set
- Sub-tree replacement pruning (WF 6.1)
  - for each node:
    - pretend remove node + all children from the tree
    - measure performance on validation set
  - remove node that results in greatest improvement
  - repeat until further pruning is harmful

Avoiding Overfitting in DTs

- **Criteria for judging which nodes could potentially be pruned**
  - Use a validation set (separate from the training set)
    - Prune each possible node that doesn't hurt the accuracy on the validation set
    - Greedily remove the node that improves the validation accuracy the most
    - Stop when the validation set accuracy starts worsening

Prepruning

Two basic approaches

1. Prepruning: Stop growing the tree at some point during construction when it is determined that there is not enough data to make reliable choices.

Advantages:

- Prevents overfitting early on.
- Faster computation as the tree is smaller.

Disadvantages:

- May miss important patterns if stopped too early.
- Decision to stop may be subjective.

Postpruning

Two basic approaches

2. Postpruning: Grow the full tree (Allow tree to grow until best fit (allow overfitting)) and then remove nodes that seem not to have sufficient evidence or or do not significantly contribute to the overall accuracy of the tree. (more popular)

Advantages:

- Captures all possible patterns.
- Reduces overfitting by removing unnecessary nodes.

Disadvantages:

- More computationally expensive initially.
- Risk of overfitting if not properly validated.

Summary: Determine when to stop splitting

Stopping Criteria:

The recursive splitting process continues until a stopping criterion is met. Common stopping criteria include

- Reaching a maximum depth (preventing it from becoming overly complex).
- Having a minimum number of samples per leaf (Splitting stops if a leaf node would contain fewer samples than a specified threshold).
- Encountering pure nodes (nodes containing only one class).

Summary

- ID3: grows decision tree from the root down
  - greedily selects next best attribute (Gain)
- Searches a complete hypothesis space
  - prefers smaller trees, high gain at the root
- Overfitting addressed by post-prunning
  - prune nodes, while accuracy ⇧ on validation set
- Can handle missing data (see WF 6.1)
- Easily handles irrelevant variables
  - Information Gain = 0 => will not be selected

Scikit Learn Decision Tree

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# Load data
iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Train a decision tree
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)

# Evaluate
print(f"Train accuracy: {clf.score(X_train, y_train):.2f}")
print(f"Test accuracy: {clf.score(X_test, y_test):.2f}")

# Visualize the tree
plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=iris.feature_names, class_names=iris.target_names)
plt.show()
```

Scikit Learn: Example with Custom Parameters:

Parameter	Description	Example Values	Effect
`max_depth`	Maximum depth of the tree.	`3`, `5`, `None`	Deeper trees → more complex (risk overfitting).
`min_samples_split`	Minimum samples required to split a node.	`2`, `5`, `10`	Higher values → simpler trees.
`min_samples_leaf`	Minimum samples required at a leaf node.	`1`, `2`, `4`	Prevents tiny leaves (noise-sensitive splits).
`criterion`	Splitting criterion (`gini` or `entropy`).	`"gini"`, `"entropy"`	`gini` (default) is faster; `entropy` may find better splits.
`max_features`	Number of features to consider for splitting (`None` = all).	`"sqrt"`, `1`, `2`	Reduces overfitting by limiting feature choices.

```
clf_tuned = DecisionTreeClassifier(
    max_depth=3,
    min_samples_split=5,
    criterion="entropy",
    random_state=42
)
clf_tuned.fit(X_train, y_train)
```

The End